UML
(Lenguaje Unificado de Modelado)

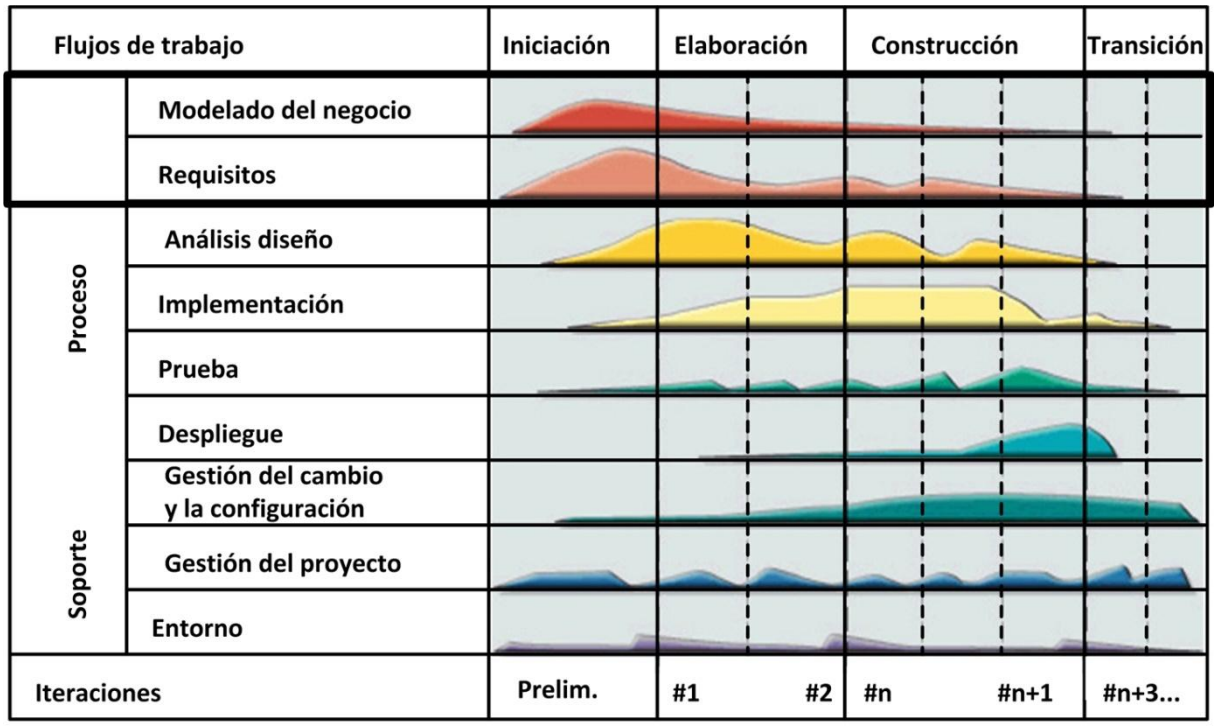
Capítulo III

Modelo de Casos de uso

1. Introduciendo el tema

Para introducirnos en el tema de los requisitos a través de casos de uso presentamos el gráfico del proceso unificado, ya mencionado en el capítulo anterior. Dicho gráfico representa una metodología de desarrollo de software que fue elaborado por los mismos creadores de UML.

En esta metodología el primero de los procesos es la de obtención de requisitos (requerimientos: se refiere a la petición que se hace de algo, se requiere o solicita; requisito: es la condición que debe cumplir algo, en general el requisito cumple con lo que se requiere o el requerimiento); y por cada uno de los procesos (requisitos, análisis, diseño, implementación, prueba y despliegue) se avanza en las fases. Es decir que cuando estoy obteniendo los requisitos realizamos actividades de las fases de inicio, elaboración, construcción y transición.

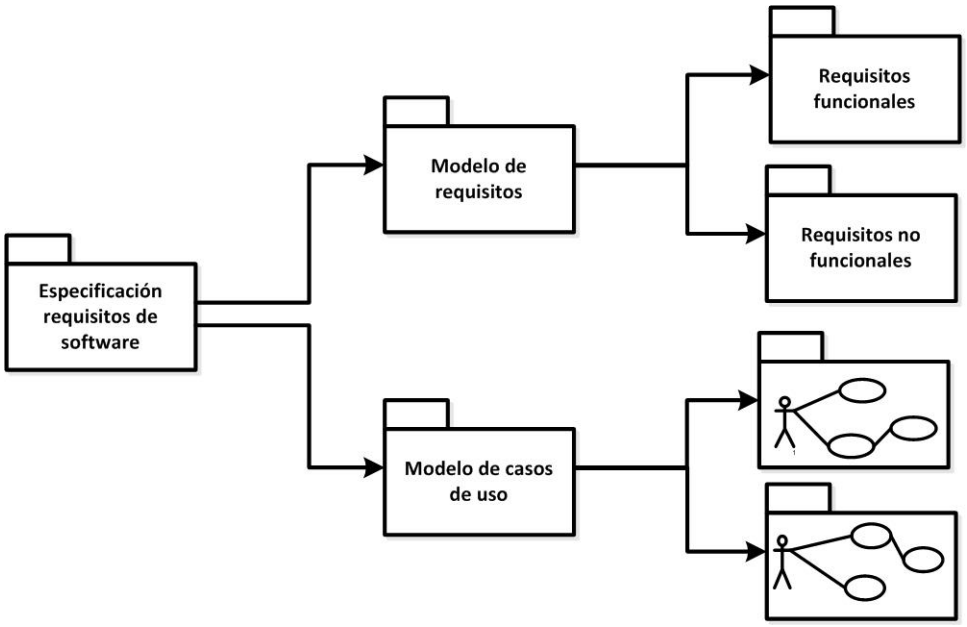


En todo trabajo de desarrollo de sistemas existe generalmente un usuario (cliente) conocido, que tiene un conjunto de necesidades que desea sean cubiertas con la construcción de un software a su medida, estos requisitos (requerimientos) pueden ser de dos tipos:

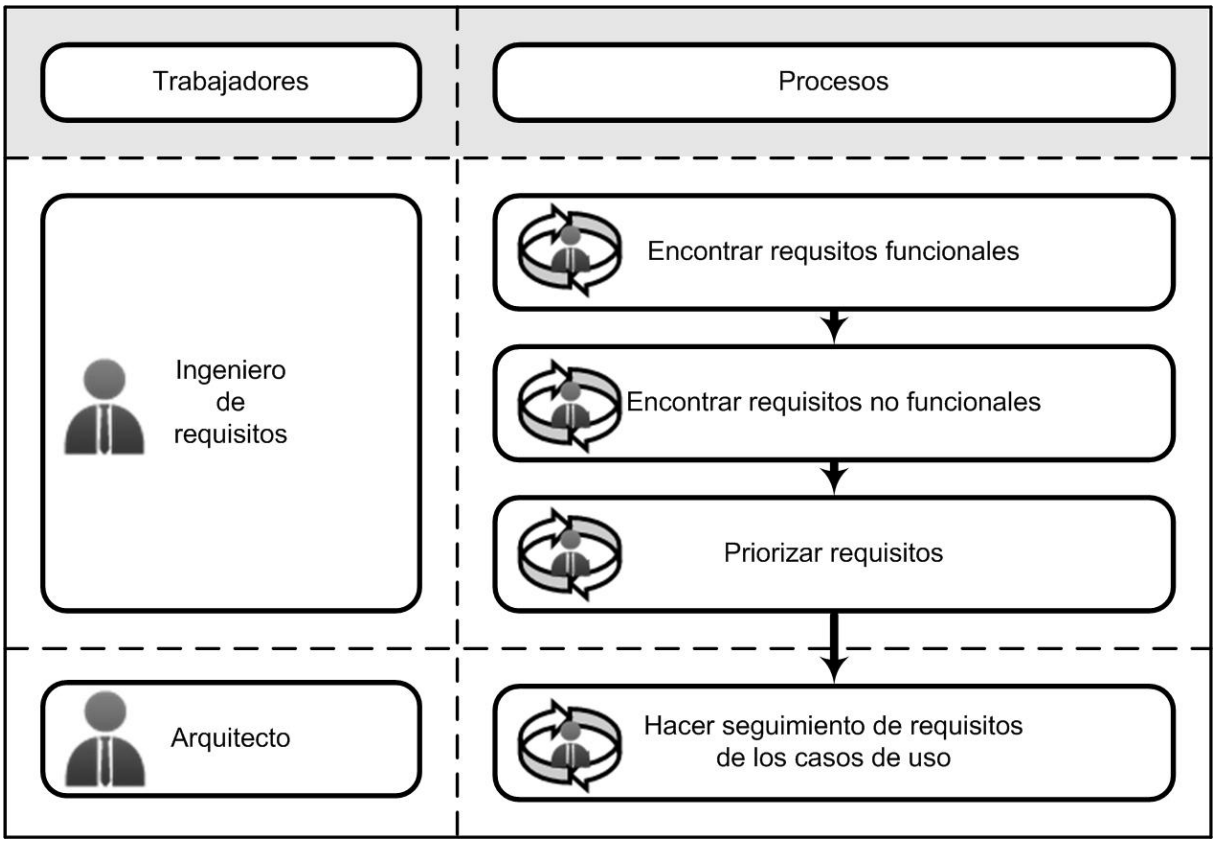
- **Funcionales:** especifican los que el sistema debería hacer. Es decir qué comportamiento debería ofrecer el sistema.
- **No funcionales:** expresan restricciones no funcionales al sistema, como por ejemplo seguridad, velocidad de respuesta, cantidad de transacciones por alguna medida de tiempo, etc. Una propiedad específica o restricción del sistema.

Los requisitos funcionales se expresan a través de diagramas de casos de uso, que a su vez se pueden agrupar en paquetes, donde cada paquete puede tener varios diagramas de casos de uso similares.

Lo mencionado anteriormente se puede ver en el gráfico siguiente.



Cuando se trata de requisitos se tiene que hablar de dos roles importantes, el de ingeniero de requisitos y del arquitecto, que serán los responsables de obtener los requisitos del cliente con los distintos métodos que existe. Esto se puede ver en la imagen y tabla siguiente.

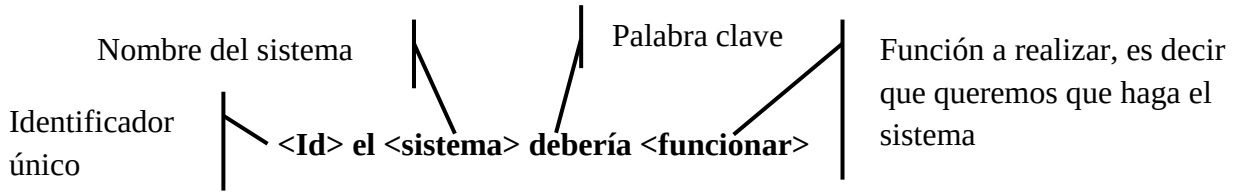


Trabajadores	Procesos
Ingeniero de requisitos	Encontrar requisitos funcionales
	Encontrar requisitos no funcionales
	Priorizar requisitos

Arquitecto	Hacer seguimientos de requisitos a casos de uso

Los requisitos son una parte muy importante de los sistemas, porque es la base de los mismos. Pero en los requisitos se debe obtener el “qué” y no el “cómo”.

Los requisitos deberían ser escritos con alguna norma para estandarizarlos, por ejemplo la siguiente:



Ejemplos de requisitos funcionales:
1 El cajero debería comprobar la validez de la tarjeta insertada
2 El cajero debería validad el pin ingresado por el usuario

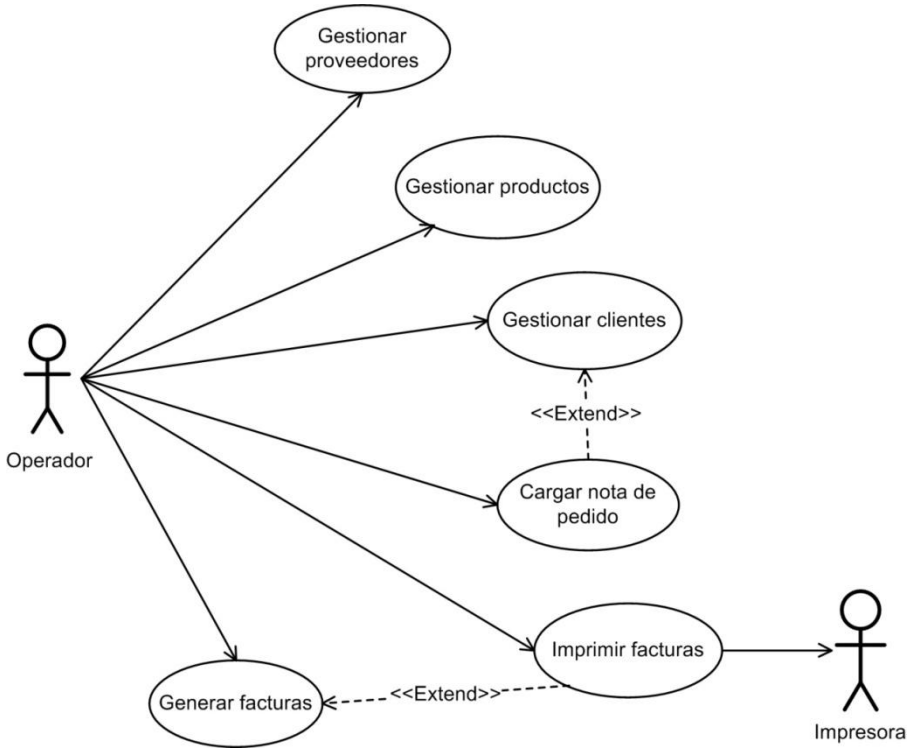
Ejemplos de requisito: no funcionales:
1 El sistema del cajero debería validar una tarjeta en menos de tres segundos
2 El cajero debería validar el pin ingresado en menos de tres segundos

Los requisitos no funcionales pueden ser de distintos tipos:
Del producto: especifican o restringen el comportamiento del software. Incluyen la usabilidad, la eficiencia (rendimiento y espacio), la confiabilidad y la seguridad del software
De la organización: estos requerimientos derivan de políticas y procedimientos de la empresa, de los clientes y de los desarrolladores. Entre ellos podemos encontrar los ambientales, los operacionales y los de desarrollo.
Externos: entre estos requerimientos podemos encontrar los regulatorios que incluye lo que es sistema debe hacer para que sea aprobado, están los requerimientos legislativos (contables y los de protección y seguridad) que son los que deben seguirse para cumplir con determinadas leyes y finalmente los éticos.

En ingeniería del software, un **caso de uso** es una técnica para la captura de requisitos de un nuevo sistema o una actualización (modificación) de software. Cada caso de uso proporciona uno o más escenarios que indican cómo debería interactuar el sistema con el usuario o con otro sistema para conseguir un objetivo específico. Es decir que existe un escenario normal y uno o más escenarios alternativos. Normalmente, en los casos de usos se evita el empleo de lenguaje técnicos, prefiriendo en su lugar un lenguaje más cercano al usuario final, este lenguaje debe ser entendido por el usuario.

Si lo decimos con otras palabras, un caso de uso es una secuencia de interacciones que se desarrollarán entre un sistema y sus actores en respuesta a un evento que inicia un actor principal sobre el propio sistema. Los diagramas de casos de uso sirven para especificar la comunicación y el comportamiento de un sistema mediante su interacción con los usuarios y/u otros sistemas. O lo que es igual, un diagrama que muestra la relación entre los actores y los casos de uso en un sistema. Una relación es una conexión entre los elementos del modelo, por ejemplo la especialización y la generalización (herencia) son relaciones. Los diagramas de casos de uso se utilizan para ilustrar los requerimientos del sistema al mostrar cómo este

reacciona a los eventos que se producen en su ámbito donde está inserto, o en él mismo sistema.



El modelo de casos de uso constituye una parte fundamental del UML (Lenguaje de Modelado Unificado). Las características principales de este modelo son las siguientes:

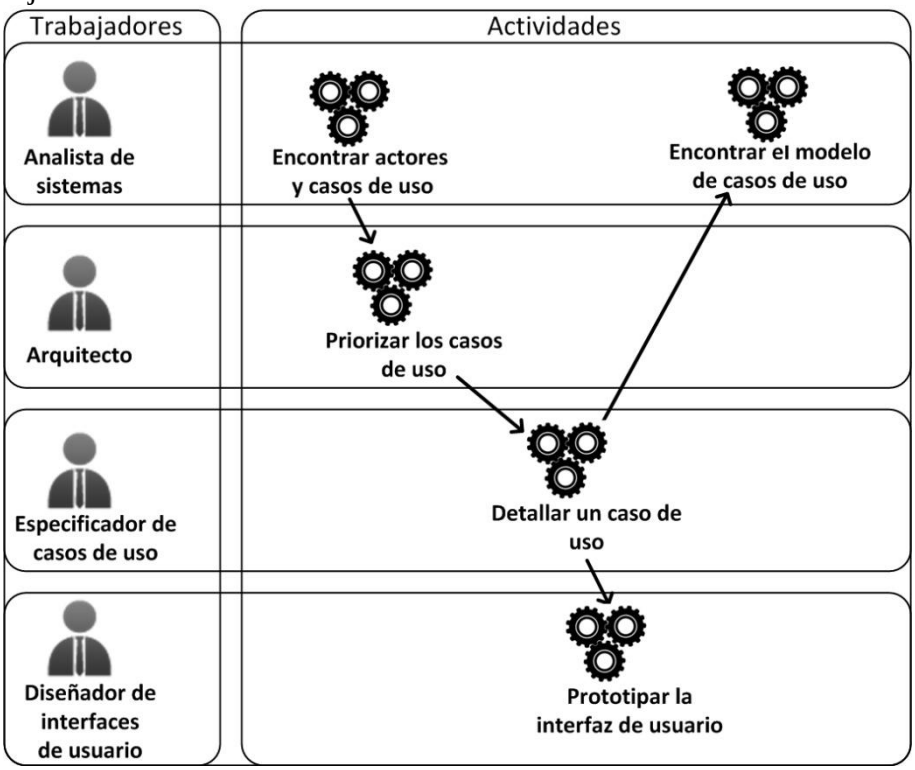
- Proporciona un medio para capturar los requerimientos funcionales de un sistema, enfocándose en el punto de vista del usuario.
  Los casos de uso son las funciones que proporciona un sistema en beneficio de sus usuarios, de esta manera, cada usuario utiliza uno o más casos de uso para llevar a cabo su trabajo. Un caso de uso vendría a ser las funciones (burbujas) en el análisis de sistemas estructurado, como lo describe Edward Yourdon, o Gane Sarsons.
- Posibilita que los desarrolladores y los clientes lleguen a un acuerdo sobre los requisitos del sistema. Una vez establecido, el modelo puede actuar como contrato entre las partes, y es la entrada principal para luego desarrollar el sistema.
- Es una importante fuente para generar la documentación de usuario del sistema en paralelo con el desarrollo. Los casos de uso proporcionan una fuente directa para la escritura de los manuales de usuario y para generar las ayudas en línea.
- Dirige el desarrollo del software, haciendo que las etapas de análisis, diseño y prueba se lleven a cabo basándose en los casos de uso.
  Una vez que se tienen los requerimientos expresados como casos de uso, se realiza el análisis del mismo, identificando las clases que participan en cada caso de uso. Además (si es necesario), se realizan prototipos de interfaces de usuario para cada caso de uso, esta interface poder ser una sola o varias, dependiendo de la complejidad del caso de uso. A medida que se avanza en el diseño, estas clases se van refinando y se describen las interacciones entre objetos que se llevan a cabo en cada caso de uso. Luego, los casos de uso se

implementan (programan) y se prueban en forma individual e integrándose con otros, para verificar si el sistema cumple con los requerimientos originales del usuario.
En síntesis, los casos de uso actúan como guías para las diferentes etapas del desarrollo, y como enlace entre los distintos modelos del sistema, brindando trazabilidad entre los mismos.

- Permiten planificar el desarrollo en iteraciones. Esto significa que se puede empezar el desarrollo por los casos de uso que se consideren más importantes, para desarrollarlos primero, y particionar los restantes en iteraciones hasta completar toda la funcionalidad proyectada para el sistema. Es decir que se puede trabajar en forma iterativa e incremental.

Al igual que como vimos en los requisitos los casos de usos tienen sus flujos de trabajo (workflows), como se ve en la imagen y tabla siguiente:

Flujo de trabajo de casos de uso



Trabajadores	Procesos
Analista de sistemas	Encontrar actores y casos de uso
Arquitecto	Priorizar los casos de uso
Especificador de casos de uso	Detallar un caso de uso
Diseñador de interfaces del usuario	Prototipar la interfaz del usuario
Analista de sistemas	Encontrar modelo de casos de uso

Como podemos ver en la tabla anterior los participantes en esta etapa son 4:

1. El analista de sistemas
2. El arquitecto

3. El especificador de casos de uso
4. El diseñador de interfaces del usuario.

Más adelante vamos a detallar más cada uno de los componentes de esta tabla.

Antes de continuar con el tema de los casos de uso queremos aclarar dos términos que usaremos mucho de aquí en adelante.

Modelo del dominio: según Jacobson, I., Booch, G. y Rumbaugh, J. (1999), el modelo de dominio "captura los tipos más importantes de objetos en el contexto del sistema, los objetos del dominio representan las "cosas" que existen o los eventos que suceden en el entorno en el que trabaja el sistema". El modelo del dominio se describe usando diagramas de UML, siendo los más utilizados los diagramas de clases, estos diagramas muestran cómo se relacionan estas clases, además muestran a los clientes, usuarios, y a los desarrolladores cuales son las clases que participan del dominio del problema y las relaciones existentes entre ellas que se manifiestan mediante asociaciones.

Para Larman, C. (2003), Un **modelo del dominio** es una representación visual de las clases conceptuales u objetos del mundo real en un dominio de interés.

Siguiendo los mismos autores, estas clases pueden aparecer de tres formas

1. Objetos de negocio que representas cosas que se manipulan en el negocio, como pedidos, cuentas y contratos.
2. Objetos del mundo real y conceptos de los que el sistema debe hacer un seguimiento, como el ejército enemigo, los misiles y sus trayectorias.
3. Sucesos que ocurrirán o han ocurrido, como la llegada de un barco, su salida y la hora que estará disponible para poder despegar.

El modelado del dominio es una variante simplificada del modelo de negocio en la cual nos centramos sólo en las cosas, es decir las clases del dominio o entidades del negocio que necesitan usar los trabajadores.

El objetivo del modelo del dominio es que los desarrolladores puedan comprender y describir cuales son las clases más importantes que componen el contexto del sistema. Este modelo está representado por las clases más importantes, luego cuando se realice el diseño del sistema se encontrarán muchas otras clases que son las clases secundarias.

Por lo dicho hasta ahora se puede resumir que el modelo de dominio debe contribuir a la comprensión del problema que el sistema resuelve en relación con su contexto.

Como ejemplo:

La empresa TodoLimpito S.A. que se dedica a la comercialización de artículos de Limpieza. Trabaja con dos tipos de clientes: mayoristas y minoristas a los cuales brinda una atención diferenciada.

Los pedidos mayoristas se analizan apenas se reciben en la empresa. Lo primero que se realiza es la verificación de los datos del cliente (y se los actualiza en caso de ser necesario), también se verifica si existen pedidos pendientes. Se suma el total del pedido (y de los pedidos pendientes, si estos existiesen), y se controla que el crédito en la cuenta corriente, establecido por la Administración, no se encuentre excedido; en ese caso se rechaza, caso contrario se revisa el stock actual para verificar si se puede cubrir el pedido actual y los pedidos pendientes.

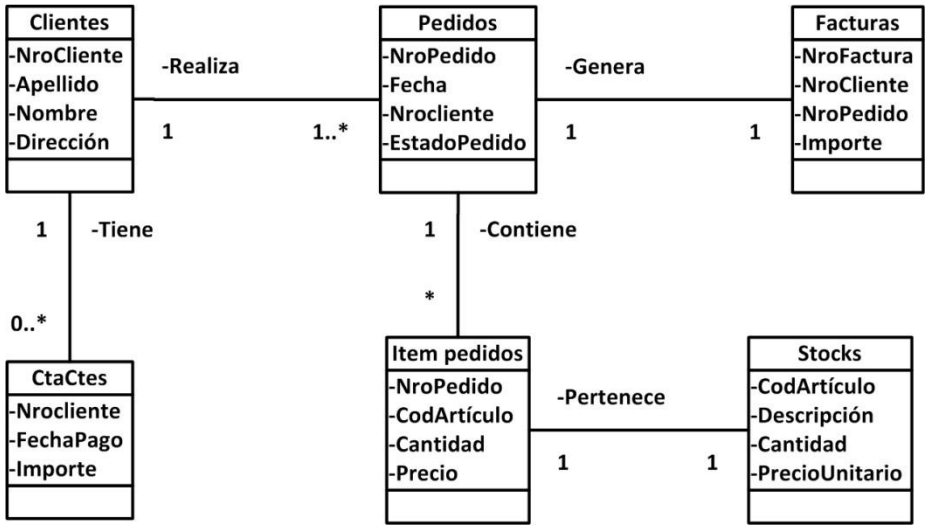
El pedido se acepta solo si se puede cumplir con el pedido completo solicitado por el cliente. Si hubiese pedidos pendientes y pueden ser satisfechos con el stock actual se adjuntan al nuevo pedido, se actualizan los pedidos aceptados. Se le responde siempre al cliente y se envía una copia del pedido a depósito para que prepare el pedido. Una vez que se verificó el stock para un pedido, los productos ya quedan reservados y descontados del stock.

Cuando el cliente mayorista llega a retirar el pedido se dirige a Caja, quién consulta los pedidos que tiene el cliente, genera las facturas (en dos copias), y realiza la cobranza. Una copia de la factura la entrega al cliente, la segunda copia la archiva. Es cliente mayorista se dirige al depósito con el original de la factura para retirar el pedido.

Los pedidos de clientes minoristas son atendidos en mostrador y son sólo al contado, se controla el stock actual (solo se venden los productos que existen en stock). Se informa al cliente los productos que existen en stock y el importe de la factura, esta se genera en forma inmediata. El original de la factura se entrega al cliente y la copia se archiva en la empresa.

El último primer día hábil de cada mes el departamento de Administración emite el listado de ventas del mes anterior, el mismo se encuentra ordenado por tipo de cliente y luego por fecha de compra.

En la figura siguiente se muestra un modelo de dominio que se ha construido a partir del enunciado anterior



Modelo del negocio: describe los procesos de negocio en términos de casos de uso del negocio y actores del negocio que se corresponden a los procesos del negocio y los clientes, respectivamente. Estos casos de usos son generales.

El modelado del negocio es una técnica para comprender los procesos de negocio de la organización.

El modelo del negocio se puede representar por dos tipos de modelos de UML:

- El modelado de casos de uso
- modelos de objetos.

Para Jacobson, I., Booch, G. y Rumbaugh, J. (1999), el "Modelo de Casos de Uso del Negocio describe los procesos de negocio de una empresa en términos de casos de uso del negocio y actores del negocio que se corresponden con los procesos del negocio y los clientes respectivamente. Al igual que el modelo de casos de uso para un sistema software, el modelo de casos de uso del negocio presenta un sistema (en este caso, el negocio) desde la perspectiva de su uso, y esquematiza como proporciona valor a sus usuarios".

Un modelo de objetos del negocio describe como cada caso de uso del negocio es llevado a cabo por parte de un conjunto de trabajadores que utilizan un conjunto de entidades del negocio y de unidades de trabajo.

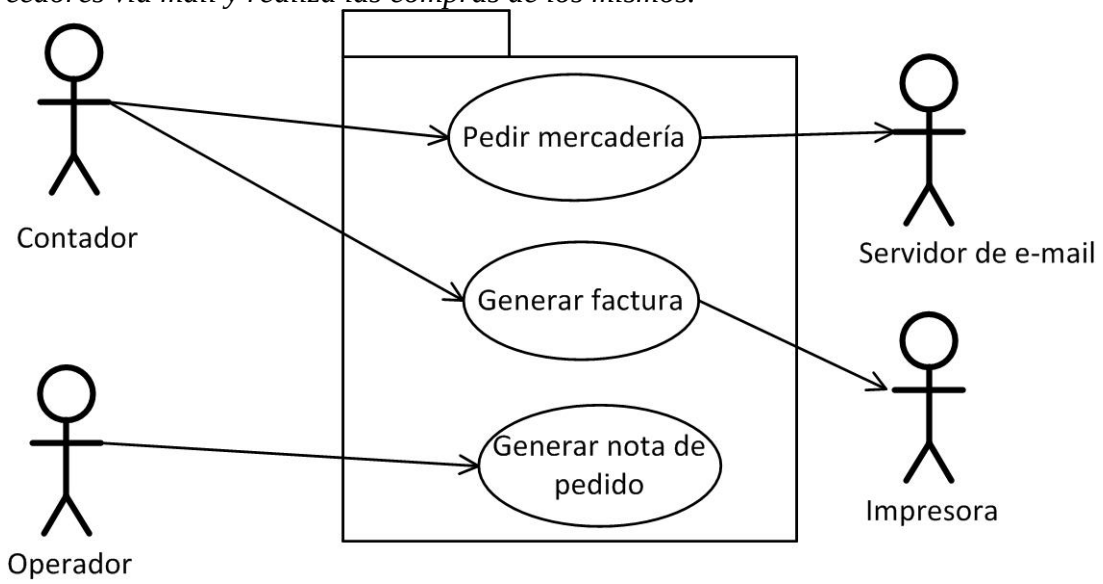
Un modelo de negocio está representado un diagrama de casos de uso, donde los actores tienen relación directa con todos los casos de uso. Es decir que no debe haber *Extend,*

Include ni *Use case* entre casos de usos. Durante el desarrollo de este capítulo nos explayaremos en estos tres conceptos (*extend*, *include* y *use case*).

Cada realización de un caso de uso del negocio puede mostrarse en diagramas de interacción y diagramas de actividad.

El diagrama del modelo de negocio tiene una forma similar en el enunciado y a la figura siguiente:

Una empresa dedicada la comercialización de productos de limpieza, la cual realiza ventas por pedidos. Los pedidos se realizan por teléfono o el cliente se acerca al negocio y realiza el pedido en forma personal. En ambos casos son atendidos por el personal de ventas de la empresa. Los pedidos recibidos son enviados al contador para que proceda a realizar las facturas que son impresas para adjuntar al pedido. En caso que lo solicitado en los pedidos no pueda ser cubierto por el stock actual, el contador se contacta con los proveedores vía mail y realiza las compras de los mismos.



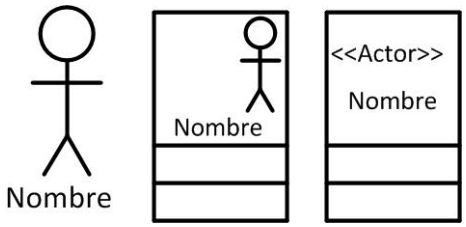
Como se puede ver en la figura anterior, los clientes y los proveedores no figuran como actores, como sería de esperar si estuviésemos desarrollando sistemas usando la metodología estructurada. Sin embargo aparece la impresora (que reemplaza al cliente destinatario de la factura) y el servidor de e-mail (que reemplaza al proveedor destinatario del pedido de compras). Estas diferencias entre las metodologías de desarrollo estructurado y la metodología orientada a objetos las veremos desarrollada a lo largo de este material.

2. ¿Cómo se compone el modelo de casos de uso?

El modelo de casos de uso está constituido principalmente por los **actores** y los **casos de uso del sistema**. Si estamos acostumbrados a la metodología estructurada, en lugar de actores usaríamos los "entes externos", sin embargo en la metodología orientada a objetos usaremos la denominación "Actores". La interacción entre los actores y los casos de uso se muestra en diagramas de casos de uso. En el modelo también se suelen incluir otros elementos como prototipos de interfaz de usuario, especificaciones suplementarias, glosario y descripción de la arquitectura.

Dado que el modelo puede hacerse grande y difícil de entender, se puede descomponer el mismo en paquetes que agrupen los casos de uso de acuerdo a algún criterio, como por ej: funcionalidad común, actores que los utilizan, etc.

A continuación se explican en detalle cada uno de estos elementos.



2.1. Actores

El actor es quién se relaciona directamente con el sistema, de forma similar a lo que hace un ente externo en la metodología estructurada. Con la diferencia que el ente externo normalmente no se relaciona en forma directa con el sistema, sino a través de alguien que opera el sistema. Claro que si el sistema es un software que se encuentra *online* y el cliente es el que opera el sistema, coincidirá el ente externo con el actor. De no ser así el actor es quién interactúa con el sistema. En caso de ser una venta por mostrador, el actor será en empleado de la empresa que maneja el sistema y no el cliente que hace el pedido. esta es la diferencia entre actor y ente externo. En la orientación a objetos se toma como actor a quién tiene una relación directa con el sistema.

Nombre: el nombre debe ser acorde al rol que representa el actor. No debe ponerse el nombre de la persona, por ejemplo no sería José, sino el rol que cumple José para el sistema, por ejemplo, comprador.

Descripción: debe ser breve, incluyendo a qué o a quién representa el actor, por qué es necesario, y qué intereses tiene en la utilización del sistema.

Características: ambiente físico en el que el actor utilizará el sistema, cantidad de usuarios representados por este actor, frecuencia con la que el actor utilizará el sistema, nivel de conocimientos, etc.

Estas propiedades, generalmente se documentan como texto asociado a la representación del actor en la herramienta CASE que se utilice.

Ejemplo:

En un sistema de un carrito de compras, se identifica un actor Comprador, con las siguientes propiedades:

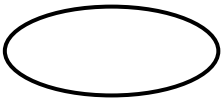
Nombre: Comprador

Descripción: Este actor es un usuario del sistema que está interesado en comprar un artículo que se encuentra disponible.

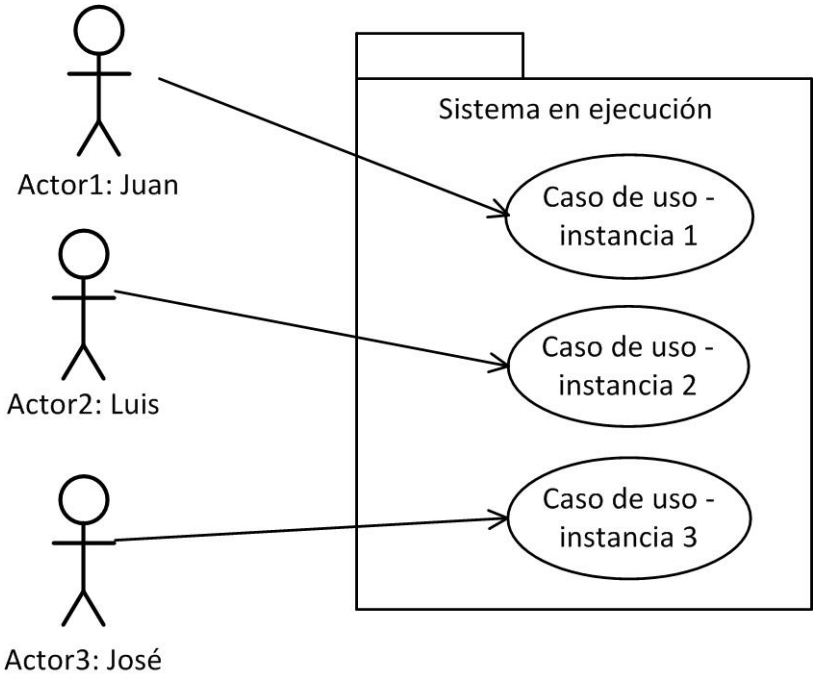
Características: el sistema se usa en un navegador en internet, estimándose unas 2350 visitas diarias.

2.2. Casos de uso

Un caso de uso es una porción (parte) de funcionalidad que el sistema ofrece para brindar un resultado de valor a sus actores. Si continuamos con las comparaciones con la metodología estructurada, un caso de uso sería una función (burbuja). Más precisamente, un caso de uso especifica una secuencia de acciones que el sistema puede llevar adelante interactuando con el actor o los actores, incluyendo distintas alternativas dentro de esa secuencia. De esta manera, un caso de uso constituye una **especificación** de una porción de la funcionalidad del sistema y está representado por el siguiente símbolo.



Cuando un usuario (una instancia de un actor) utiliza un caso de uso, se crea una instancia del mismo, que ejecuta la secuencia de acciones descripta en el caso de uso.



2.2.1. Propiedades de los casos de uso

Nombre: se debe asignar un nombre que lo identifique. Este nombre debe reflejar el objetivo de la interacción entre el actor y el sistema. Normalmente el nombre empieza con un verbo.

Ejemplos: Realizar oferta, Analizar catálogo, Agregar cliente, Eliminar cliente, Imprimir factura, etc.

Descripción: breve descripción del propósito del caso de uso y la utilidad que tiene desde el punto de vista de los actores.

Precondiciones: condiciones que deben cumplirse o darse para que pueda ocurrir la ejecución del caso de uso.

Flujos de eventos: descripciones paso a paso de QUÉ hace el actor y QUÉ hace el sistema para llevar adelante la funcionalidad del caso de uso, incluyendo desde su inicio hasta su finalización, teniendo en cuenta las condiciones normales y anormales o alternativas que puedan darse.

Requerimientos especiales: descripción de los requerimientos no funcionales relacionados con el caso de uso, a tener en cuenta en las actividades posteriores de análisis y diseño del sistema.

Postcondiciones: condiciones que se dan luego de la ejecución del caso de uso. Es decir el estado en que queda el sistema como resultado de la ejecución del caso de uso.

Puntos de extensión: lista de lugares dentro del **Flujo de eventos** donde se pueda agregar funcionalidad adicional mediante relaciones tipo "*extend*" o "*Include*".

Observaciones y datos: toda información que puede ser de utilidad se debe escribir en este lugar, como por ejemplo atributos que se han descubierto.

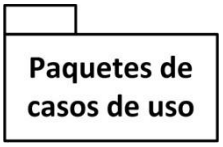
Caso de uso: baja empresa de transporte			**Nro.** CU12
Actor: usuario			**Versión:** 1
Descripción: elimina una empresa de transporte			
Activación: el usuario selecciona la opción "Borrar transporte" del menú "Administración"			
Precondición: que exista al menos una empresa de transporte cargada en el sistema y que no haya deuda o envíos pendientes			
Flujo normal		**Flujo alternativo**	
1	el usuario selecciona la opción "Borrar transporte" del menú "Administración"		
2	Se carga una página con una lista de las empresas de transporte existentes en el sistema y se selecciona la empresa a borrar	2.1	El sistema llama al caso de uso "Verificar pedidos pendientes de envío".
3	El usuario presiona el botón "Eliminar"	3.1	El usuario cancela la operación cerrando la ventana o presionando el botón "Volver".
4	El sistema pide confirmación		
5	El usuario confirma la operación	5.1	El usuario cancela la operación y se regresa al punto 2 del flujo normal
6	La empresa se elimina del sistema	6.1	Hay deudas con la empresa o pedidos pendientes, la empresa no se elimina, se muestra un mensaje de error y se regresa al punto 2 del flujo normal
Post-condiciones:	Éxito: se elimina la empresa de transporte		
	Fracaso: la empresa permanece cargada		
Puntos de extensión: en el paso 2 el usuario puede llamar al caso de uso "Verificar pedidos pendientes de envío".			
Requisitos especiales:			
Observaciones y datos:			

Alternativamente, los flujos de eventos de un caso de uso se pueden documentar utilizando diagramas de actividades. A pesar de ello, el medio más difundido y de preferencia es el texto, por ser más simple y fácil de entender. Cuando nos referimos a texto estamos haciendo referencia a la tabla anterior.

2.3. Paquetes de casos de uso

Los paquetes de casos de uso nos permiten agrupar casos de uso que tengan funciones afines (funcionalidad común, actores que los utilizan, etc.). Se utilizan cuando la cantidad de casos de uso en un sistema se hace significativa (más de 20), de manera de estructurar el modelo dividiéndolo en partes más pequeñas. Por ejemplo en un sistema de producción de una fábrica podríamos agrupar los casos de usos en los paquetes de: compras,

producción y ventas.



Algunos criterios a tener en cuenta para agrupar casos de uso en paquetes, pueden ser los siguientes:

- Casos de uso que forman parte de un mismo proceso del negocio.
  Ejemplo: en un sistema de información de una empresa que comprenda varios procesos, se pueden agrupar todos los casos de uso vinculados al proceso de compras en un paquete, el de facturación en otro paquete de casos de uso.
- Casos de uso que comparten una funcionalidad común dentro del sistema.
  Ejemplo: en un sistema se identifican casos de uso para agregar, modificar, eliminar y localizar clientes y otros para agregar, modificar, eliminar y localizar productos. El primer grupo se puede incluir en un paquete Administración de clientes y el segundo en un paquete Administración de productos.
- Casos de uso utilizados por el mismo actor o grupo de actores.
  Ejemplo: en un sistema de información, se tiene un conjunto de casos de uso que utilizan los vendedores y otros que utilizan los administradores. El primer conjunto se puede incluir en un paquete Vendedores y el segundo en un paquete Administración.

2.4. Diagramas de casos de uso

Los diagramas de casos de uso son diagramas donde se muestran los actores y los casos de uso de un sistema (o de un paquete, porción de un sistema), y cuáles son las relaciones que existen entre ellos. Los actores, casos de uso y paquetes se representan respectivamente con los símbolos mostrados en las figuras siguientes.

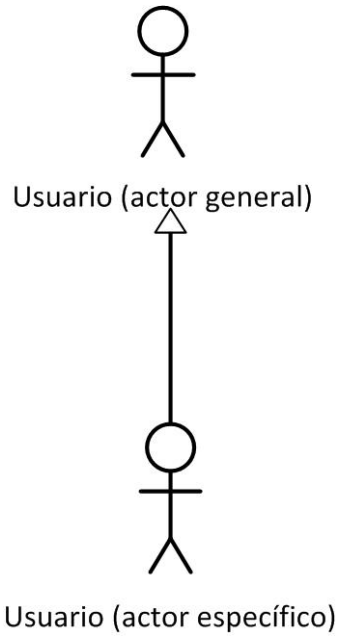
Estos diagramas tienen la utilidad de servir como mapas donde se puede apreciar la funcionalidad del sistema completo, los actores del sistema y los casos de uso que utilizan cada uno de ellos, la forma en que se relacionan los casos de uso entre sí, y la distribución en paquetes (grupos funcionales) en el caso de sistemas grandes.

Las relaciones que pueden existir entre los elementos de un diagrama de casos de uso son las que se explican en los apartados siguientes.

2.4.1. Relaciones entre actores

La única relación posible entre actores es la **generalización**. Dado que los actores son clases, la generalización tiene el mismo significado y representación que la generalización entre clases en UML, esto se denomina **herencia**.

Representación de la relación de generalización entre actores en UML. En el ejemplo se puede ver que existe un actor general (Usuario) que representa a todos los usuarios del sistema, y hay un actor más específico que representa a los usuarios que cumplen el rol específico en el sistema.

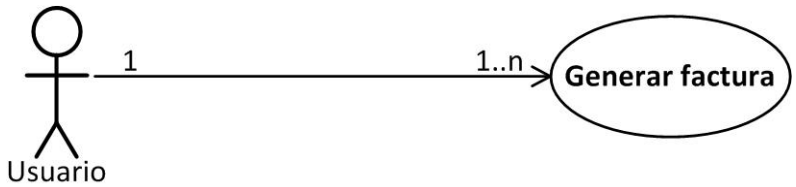


2.4.2. Relaciones entre actores y casos de uso

La única relación posible entre los actores y los casos de uso es la **asociación de comunicación**. Esta asociación representa las señales que se envían mutuamente los actores y los casos de uso en su interacción. Estas asociaciones se pueden comparar a los flujos de datos que existen entre una unidad externa y una función (burbuja) en la metodología estructurada.

Las asociaciones de comunicación tienen un **sentido o navegabilidad**, que indica cuál de los extremos (el actor o el caso de uso) es el que inicia y comanda la interacción.

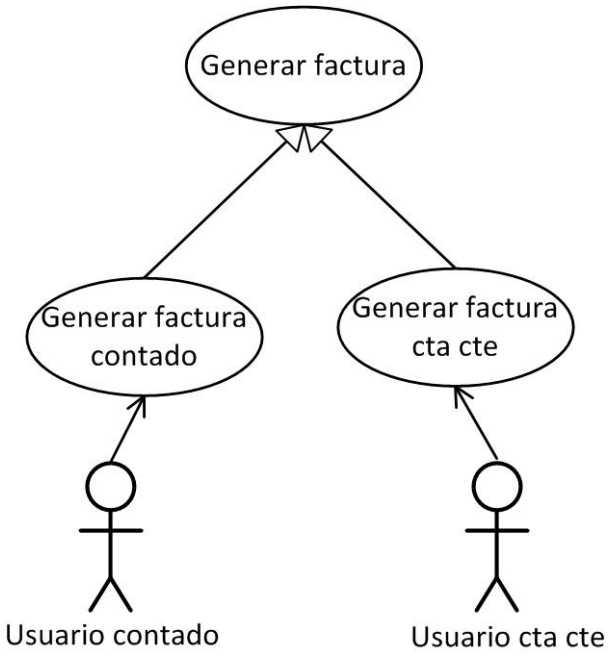
Opcionalmente pueden tener una **multiplicidad,** indicando cuántas instancias de un extremo se asocian con las del otro extremo.



2.4.3. Relaciones entre casos de uso

Existen tres relaciones posibles entre los casos de uso: **generalización, inclusión, y extensión.**

Generalización: La relación de generalización se utiliza para hacer que un caso de uso sea una especialización de otro. Cuando se encuentra que varios casos de uso tienen gran parte de su comportamiento en común, se puede agrupar ese comportamiento común en un caso de uso padre, haciendo que los demás estén relacionados con el mismo a través de relaciones de generalización.

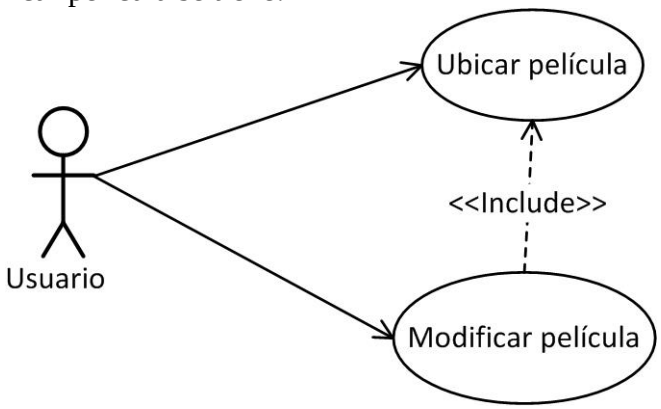


De la relación de generalización entre casos de uso en UML. En el ejemplo se puede ver que los casos de uso Generar factura contado y General factura cta. cte. son especializaciones del caso de uso Generar factura. Es decir hay una parte común entre generar factura contado y generar factura en cta. cte. Que se realiza en el caso de uso generar factura y las particularidades de cada tipo de factura en los casos de uso de factura de contado y cta. cte.

Inclusión: La relación de inclusión se utiliza para indicar que un caso de uso incluye a otro como parte de su flujo de eventos. Esto significa que el comportamiento del caso de uso incluido se inserta explícitamente dentro del comportamiento del caso de uso que lo incluye.

Para la representación de la relación de inclusión entre casos de uso en UML. En el ejemplo se puede ver que el casos de uso Modificar o inhabilitar película incluye al caso de uso Localizar película.

Esto significa que dentro de alguno de los caminos del flujo de eventos de Modificar o inhabilitar película, el usuario utiliza el caso de uso Ubicar película. En la documentación del caso de uso Modificar película se tiene:

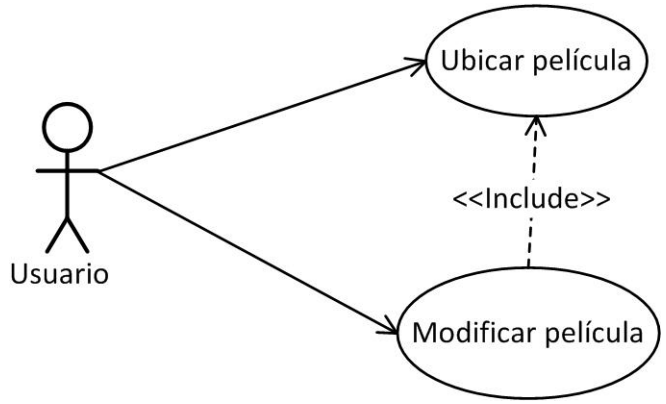


...
Flujo principal (de la explicación del caso de uso)
1. El usuario utiliza el caso de uso Localizar película y selecciona la opción “Modificar”
2. El sistema muestra los datos de la película que pueden ser modificados y habilita la opcón

de modificación.
...

Esta relación también permite compartir comportamiento común entre casos de uso, pudiendo reemplazar en cierto modo al uso de la generalización. Cuando se utiliza inclusión, quiere decir que estoy usando siempre ese caso de uso que llamo con la asociación *include*, pero tiene sentido llamarlo con un *include* cuando algún otro caso de uso también usará ese caso de uso como un *include* o un *extend*, de lo contrario no tiene sentido separarlo en un caso de uso para luego llamarlo. Siguiendo el ejemplo, cuando se utiliza el caso de uso modificar o inhabilitar película, siempre se necesita llamar al caso de uso localizar película, entonces se usa un *include*, porque siempre se necesita localizar una película antes de modificarlo o inhabilitarla, pero si no existiese ningún otro caso de uso que utilice localizar película, no tendría sentido que se lo llame con un *include*, sino que convendría incluirlo como parte del caso de uso modificar o inhabilitar película. Esto da a entender que no hay que crear casos de usos con porciones de funcionalidad sin tener una justificación del motivo porque se lo hace.

Extensión: La relación de extensión se utiliza para anexar comportamiento opcional a un caso de uso. Generalmente se utiliza para vincular casos de uso de manera que mientras se ejecuta uno, opcionalmente se puede ejecutar otro como parte de su flujo de eventos.
Es similar al *include*, pero este caso de uso que se llama por un *extend* es opcional.



Representación de la relación de extensión entre casos de uso en UML. En el ejemplo se puede ver que el casos de uso Imprimir códigos de película extiende al caso de uso Agregar película.

Esto significa que en algún punto dentro del flujo de eventos de Agregar película, el usuario tiene la posibilidad de utilizar opcionalmente el caso de uso Imprimir códigos de barra de la película.

En la documentación del caso de uso Agregar película se tiene:

...

Puntos de extensión

En el paso n del flujo principal se puede tener como extensión el caso de uso Imprimir código película.
...

Como observación general, se puede decir que no es conveniente abusar de las relaciones entre casos de uso, dado que tienden a complicar el modelo pudiendo llegar a hacerlo un entrecruzamiento de flujos y transformarse en inentendible para los usuarios finales o de los clientes interesados en el sistema.

2.5. Prototipos de interfaz de usuario

Los prototipos de interfaz de usuario se utilizan como ayuda para comprender y especificar las interacciones entre los actores humanos y el sistema durante la captura de requisitos.

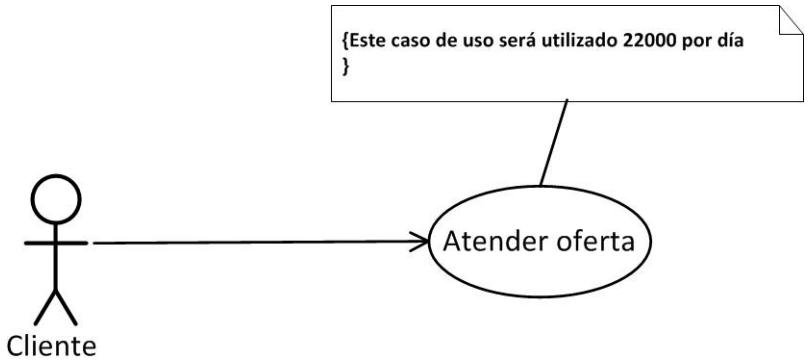
Resultan sumamente útiles para estabilizar los requerimientos, ya que a los clientes les resulta mucho más sencillo tomar decisiones o hacer especificaciones sobre algo que están viendo y/o usando que explicarlas en palabras.

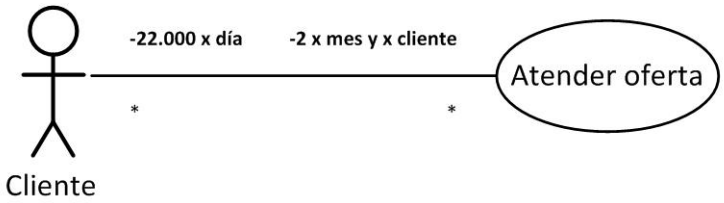
UML no contempla ningún diagrama en especial para estos fines, pudiéndose utilizar la herramienta que se considere adecuada.

2.6. Especificaciones suplementarias

Las especificaciones suplementarias (especificaciones no funcionales) se pueden documentar de varias maneras en UML. Estas son:

a) Utilizar **notas** dentro de los diagramas de casos de uso, como muestra el ejemplo de la figura siguiente





Ejemplo de documentación de requerimientos no funcionales mediante notas.

b) Utilizar la sección “Requerimientos especiales” de la documentación de cada caso de uso, por ejemplo:

Requerimientos especiales

Usabilidad: el usuario debe poder llegar a las operaciones por las que ingresó al sistema como máximo en tres clics de mouse luego de haberle logueado al sistema.

c) Agrupar todas las especificaciones suplementarias en un único documento, titulado “Especificaciones suplementarias” o “Requisitos especiales”.

2.7. Glosario

El glosario es un documento en el cual se van compilando todos los términos comunes del dominio del problema, con una breve descripción. Recuerde, es muy importante que este glosario se encuentre ordenado alfabéticamente.

2.8. Descripción de la arquitectura

Si bien UML no propone nada al respecto, algunas metodologías como el Proceso de Desarrollo Unificado (*Unified Development Process*) proponen documentar las partes más significativas de los distintos modelos (modelo de casos de uso, de análisis, de diseño y de implementación), en un documento único llamado "Documento de arquitectura del software" (en inglés, "*Software Architecture Document*").

La idea de este documento es condensar las partes más representativas de cada modelo, de manera de brindar un resumen que permita entender rápidamente el sistema completo.

El documento se estructura en distintos capítulos, donde cada uno de ellos cubre una vista del sistema.

Las distintas vistas que componen el modelo son:

- Vista lógica
- Vista de los procesos
- Vista de implementación
- Vista de despliegue
- Vista de casos de uso o de escenarios

Para más detalle ver el punto 4.3 del capítulo II.

Los casos de usos hacen su aporte a la **Vista de caso de uso,** esta vista captura los requisitos básicos para el sistema como un conjunto de casos de uso. Estos casos de uso proporcionan la base para la construcción de otras vistas.

3. Actividades para la construcción del modelo

La primera actividad para la construcción del modelo de casos de uso es la **Identificación** de **actores y casos de uso**, en la cual se identifican los actores (quienes interactúan directamente con el sistema) y casos de uso (lo que hace el sistema para responder a los actores), para tener una primera versión del modelo. Con esta primera versión del modelo se está en condiciones de realizar una estimación inicial del esfuerzo que tomará la construcción del sistema, y se tiene un panorama mucho más claro de la funcionalidad que tendrá el mismo y de su contexto. Esta primera versión es el **Modelo de negocio**.

Luego de tener el modelo de negocio, es conveniente realizar una **Priorización de casos de uso**, en esta tarea se identifican los casos de uso más importantes desde el punto de vista de la arquitectura. La priorización de los casos de uso resulta útil cuando los sistemas son medianos o grandes y es fundamentalmente para planificar el desarrollo.

Una vez que se han priorizado los casos de uso (o a medida que se va realizando la priorización), se efectúa la **Especificación de casos de uso,** en la cual se documentan detalladamente los casos de uso priorizados, ver tabla del punto 2.2.1.

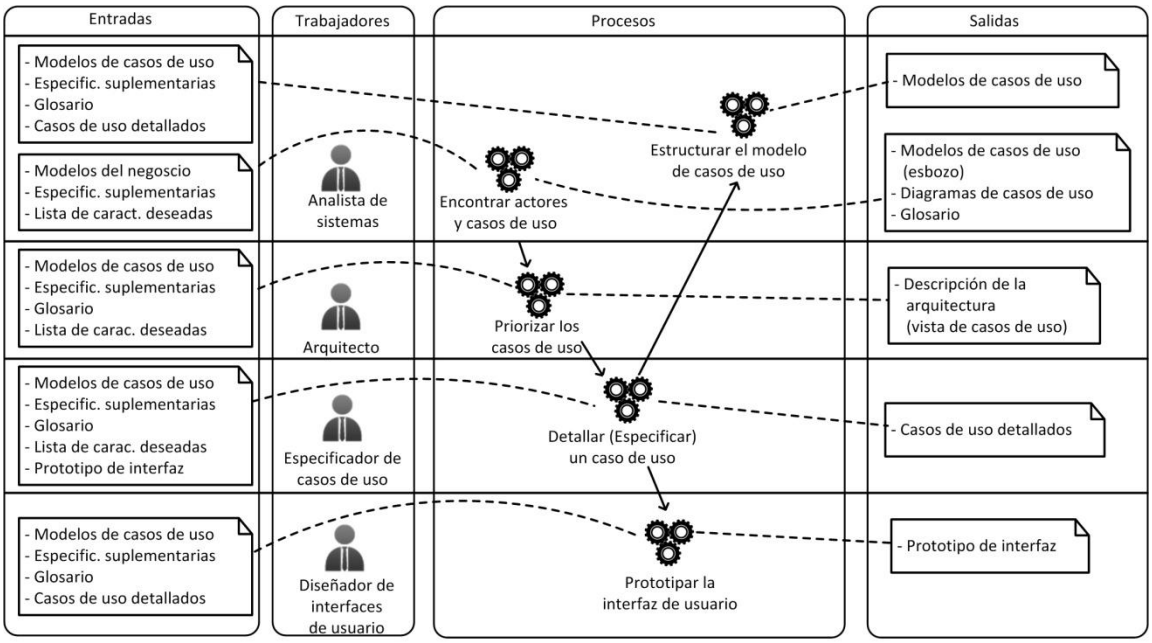
En paralelo con la especificación, se puede realizar el **Prototipado de la interfaz de usuario**, donde se construyen prototipos de interfaz, y se analizan con los usuarios o clientes para estabilizar los casos de uso y modificar o encontrar nuevos requisitos.

Durante las actividades anteriores, se pueden descubrir nuevos actores, casos de uso, relaciones, etc. Debido a esto, se efectúa una **Estructuración del modelo**, en la que se definen las relaciones de inclusión, extensión y generalización entre casos de uso, o de generalización entre actores. Debemos recordar que cuando trabajamos con la metodología del Proceso Unificado la construcción del sistema es iterativa e incrementa.

Es importante notar que las actividades de priorización, especificación, y prototipado apuntan a un enfoque iterativo para el desarrollo del sistema. La idea es seleccionar los casos de uso más significativos y continuar con las actividades siguientes de análisis, diseño

e implementación sobre estos casos de uso en lugar de especificar el sistema completo inicialmente como se haría en un enfoque en cascada.

En la imagen y la tabla siguientes se puede apreciar un diagrama de actividades UML en el que se resumen todas las actividades mencionadas con sus productos de entrada y de salida.



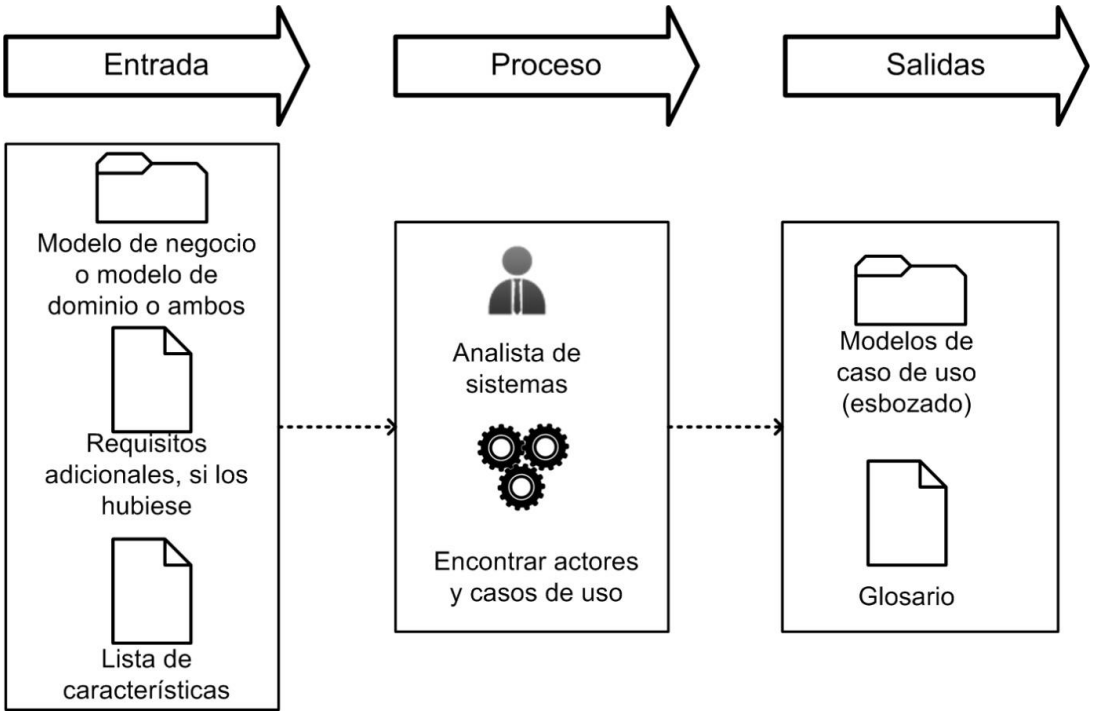
Entradas	Trabajadores	Procesos	Salidas
• Modelo de negocio • Especificaciones suplementarias • Lista de características deseadas	Analista de sistemas	Encontrar actores y casos de uso	• Modelo de casos de uso • Glosario
• Modelo de casos de uso • Especificaciones suplementarias • Glosario • Lista de características deseadas	Arquitecto	Priorizar los casos de uso	• Descripción de la arquitectura •
• Modelo de casos de uso • Especificaciones suplementarias • Glosario • Lista de características deseadas • Prototipo de interfaz del usuario	Especificador de casos de uso	Detallar un caso de uso	• Casos de uso detallados
• Modelo de casos de uso • Especificaciones suplementarias • Glosario • Casos de uso detallados	Diseñador de interfaces del usuario	Prototipar la interfaz del usuario	• Prototipos de interfaz
• Modelo de casos de uso • Especificaciones suplementarias • Glosario • Casos de uso detallados	Analista de sistemas	Encontrar modelo de casos de uso	• Modelo de casos de uso

A continuación detallaremos cada una de las actividades de la tabla anterior, teniendo en cuenta lo que necesitamos para la actividad, la actividad propiamente dicha y que obtendremos como resultado de la misma.

3.1. Identificación de actores y casos de uso

Los objetivos principales de esta actividad son los siguientes:

- Delimitar el sistema de su entorno (poner los límites), definiendo qué va a ser manejado dentro y qué va a ser manejado fuera del sistema. (En UML 2 esto se conoce como el sujeto del sistema.)
- Definir qué y quienes (actores) van a interactuar con el sistema, y qué funcionalidad (casos de uso) se espera del sistema.
- Capturar y definir un glosario de términos comunes esenciales para la creación de las descripciones detalladas de las funcionalidades del sistema (casos de uso).



Entradas	Trabajador / Proceso	Salidas
Modelado de negocio o modelo de dominio o ambos	Analista de sistemas: Encontrar actores y casos de uso	Modelo de casos de uso (esbozado)
Requisitos Adicionales, si los hubiese		Glosario
Lista de características		

La identificación de actores y casos de uso es la actividad más importante para obtener adecuadamente los requisitos, y es responsabilidad de los analistas, (ver tabla anterior).

De acuerdo al proyecto (tamaño) y al grado de entendimiento que tengan los analistas sobre el dominio, pueden existir varias formas posibles de concretar esta actividad.

Puede darse el hecho de que exista un **modelo del negocio** que servirá de punto de partida. En este caso, los analistas pueden preparar un primer borrador del modelo de casos de uso, basándose en la información contenida en el modelo del negocio.

Otras veces puede ocurrir que solamente exista una breve **descripción general**, una lista de requerimientos, o una especificación de requisitos detallada que incluya las características generales que se requieren del sistema.

También es posible que no exista ninguno de los elementos mencionados en los párrafos anteriores, situación relativamente frecuente. Independientemente de cuanta

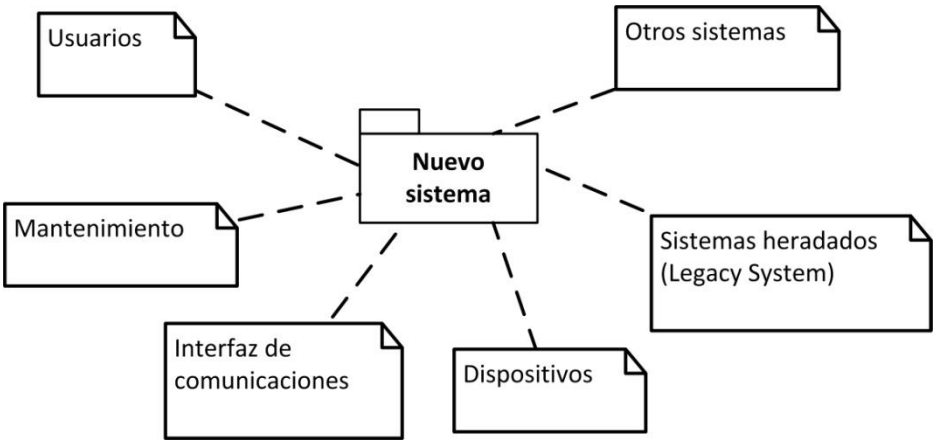
información se tenga disponible como entrada para la actividad, la misma consta de las siguientes subactividades:

- Encontrar los actores
- Encontrar los casos de uso
- Organizar el modelo de casos de uso

Estas sub-actividades normalmente se ejecutan simultáneamente. A medida que se encuentran los actores y los casos de uso, estos se modelan y describen en diagramas de casos de uso. Para realizar esta tarea es recomendable poseer una herramienta CASE, lo cual permite la revisión y edición rápidas del modelo, estas herramientas existen en versiones libres (más limitadas) y en versiones pagas, que poseen muchos más opciones de manejo y administración. Finalmente, se pueden generar reportes con todos los actores, casos de uso y sus descripciones, el cual sirve para efectuar revisiones grupales del modelo.

3.1.1. Encontrar los actores

Para esta actividad, se deben tener en cuenta todas las entidades externas al sistema que interactúen de alguna manera con el mismo en forma directa o indirecta, sean usuarios, dispositivos u otros sistemas.



Para encontrar los actores del sistema se proponen algunas preguntas clave que nos serán de gran utilidad y son las siguientes:

- ¿Qué grupos de usuarios necesitan utilizar el sistema para llevar a cabo sus tareas?
- ¿Qué grupos de usuarios utilizan las funciones principales y más obvias del sistema?
- ¿Qué grupos de usuarios utilizan las funciones secundarias del sistema tales como administración y/o mantenimiento?
- ¿El sistema interactuará con algún dispositivo de hardware?
- ¿El sistema interactuará con algún otro sistema?

Algunos criterios útiles para verificar los actores identificados:

Primero, debería ser posible identificar al menos dos usuarios (personas) que puedan representar al actor (ser instancias del actor). Si esto no es posible de verificar, el actor identificado no está representando un rol sino una persona u objeto particular, es decir que esa clase no tiene instancias.

Segundo, la coincidencia en los roles que cumplen dos actores diferentes con respecto al sistema debería ser mínima. Esto apunta a evitar tener actores que tienen

prácticamente el mismo rol ante el sistema. Cuando éste es el caso, se debe tratar de reunir ambos actores en uno solo.

Tercero, los actores deben comunicarse en forma directa con el sistema, sin intermediarios. A veces es común pensar como actores a los beneficiarios de un sistema, en lugar de a sus propios actores. Por ejemplo, en un sistema donde un **Usuario** se comunica telefónicamente con un **Operador** para solicitar un servicio, el actor del sistema es el **Operador,** ya que es quien lo utiliza en forma directa. Distinto sería el caso en que el sistema recibiera órdenes en forma telefónica, donde el **Usuario** podría ser el actor, utilizando el sistema directamente desde su teléfono, como se puede apreciar en distintos *call center* que se le solicita al usuario que elija una serie de opciones.

Una vez que se tienen los actores identificados, se modelan los mismos en un diagrama de casos de uso y cada uno de ellos se documenta como se vio en el apartado 2.1 de este capítulo.

Al principio puede ser dificultoso encontrar los actores más adecuados, y es poco probable que se puedan identificar todos los actores existentes. Posteriormente, al identificar los casos de uso, se obtiene un entendimiento más profundo del sistema, lo que da la posibilidad de refinar el modelo, agregando, modificando, reagrupando o eliminando actores.

Normalmente hay una tendencia a identificar actores en exceso al principio del análisis.

Advertencia para tener en cuenta: se debe tener cuidado al cambiar los actores, ya que este cambio puede implicar una alteración de las interfaces lo que conlleva a un cambio en el comportamiento del sistema. También es bueno recordar que el sistema actúa en beneficio de sus actores, lo que significa que si un actor desaparece porque lo eliminamos, es posible que pudiera desaparecer una parte del sistema.

3.1.2. Encontrar los casos de uso

Una vez que se tiene la mayoría los actores o a todos, se debe proceder a identificar los casos de uso del sistema.

Los primeros casos de uso que se identifican pueden no ser lo suficientemente claros, y normalmente requieren varios cambios hasta su versión definitiva. El analista debe estar preparado para agregar, eliminar, combinar y dividir los casos de uso antes de lograr una versión final de los mismos.

Una manera efectiva de encontrar casos de uso es analizar lo que cada uno de los actores identificados hasta el momento requiere del sistema. De esta manera se confirma que el sistema existe para sus usuarios, y por lo tanto debe estar planteado con el objetivo de cubrir los requerimientos de los mismos.

Para cada actor (sea este humano o no), es conveniente plantear las siguientes preguntas:

- ¿Cuáles son las principales funciones que el actor desea que el sistema lleve a cabo?
- ¿El actor: almacenará, creará, eliminará, cambiará, o leerá datos en el sistema? (altas, bajas, modificaciones y consultas)
- ¿El actor deberá ingresar información al sistema acerca de algún evento u acontecimiento que ha ocurrido fuera de los límites del sistema?
- ¿El actor debe recibir información sobre algún acontecimiento interno del sistema?
- El actor realizará alguna tarea de arranque, parada o mantenimiento del sistema?

Al responder a estas preguntas podremos identificar casos de uso candidatos.

Aunque no todos ellos se convertirán en casos de uso independientes. Al estudiar y describir los flujos de eventos en cada uno de ellos, se podrá determinar cuáles son casos de uso y cuales son secuencias de actividades dentro de un caso de uso.

A tener en cuenta: criterios importantes a tener en cuenta en la identificación de casos de uso:

- Se debe poner mucha atención al concepto de que un caso de uso debe brindar un resultado de valor al actor que lo utiliza.
- Es común identificar como casos de uso secuencias de actividades que en realidad no forman un caso de uso en sí mismas, sino una parte de una secuencia más importante, que es la que sí brinda un resultado de valor.
  Ejemplo: un caso de uso "Ingresar al sistema", normalmente no tiene sentido en sí mismo, ya que el objetivo del usuario no es ingresar al sistema, sino efectuar alguna operación de valor dentro del mismo, como por ejemplo "Realizar una consulta". El ingreso al sistema es un paso que el usuario debe efectuar para poder hacer esa consulta.
- También es importante saber que no todos los casos de uso representan pantallas o interfaces gráficas de un sistema. Por lo tanto es un error muy común pensar que un caso de uso es una pantalla y describir al mismo como una interacción entre botones, menús desplegables, cajas de texto, Menú desplegable, etc. Un caso de uso es una especificación funcional que posteriormente puede mapearse a una o más interfaces de usuario, e inclusive puede no estar asociado a ninguna interfaz de usuario.

Cada caso de uso se debe documentar, de acuerdo a lo visto en el apartado 2.2. Generalmente, los analistas efectúan una primera pasada identificando los casos de uso, asignándole un **nombre** y una breve **descripción** a cada uno de ellos. Luego, en una segunda pasada describen el **flujo principal de eventos.**

Es importante que a medida que se van documentando los casos de uso, se identifiquen y documenten los términos que forman parte del dominio del problema en un Glosario (ver apartado 2.7). Asimismo, también es importante identificar y documentar los **requerimientos especiales** (no funcionales) dentro de los casos de uso o en un documento separado (ver apartado 2.6).

Los casos de uso se documentan en diagramas de casos de uso, de la misma manera que los actores.

Es importante que los casos de se expliquen en forma bien detallada. De esta manera el programador entenderá mejor el problema y el responsable de crear el manual del usuario tendrá más allanada su tarea.

3.1.3. Organizar el modelo de casos de uso

Para realizar la organización del modelo primeramente se debe efectuar una primera estructuración de los diagramas de caso de uso teniendo en cuenta los siguientes puntos:

- Describir las relaciones de comunicación (asociaciones) entre los actores y los casos de uso
- Agrupar los casos de uso similares en paquetes, cuando el número es significativo, de acuerdo a los criterios citados en el apartado 2.3 (como guía general, resulta conveniente usar paquetes cuando los casos de uso superan el umbral de 25)
- Actualizar los diagramas de casos de uso ante cualquier cambio.

- Generar un **reporte del modelo**. Las herramientas CASE proveen la facilidad de generar estos reportes. De lo contrario deberá usar otra herramienta para crear esta documentación.

El reporte del modelo se utiliza para validar el modelo de casos de uso inicial con los involucrados en el proyecto (clientes, usuarios, desarrolladores, directivos, etc.). Esta validación apunta a determinar si:

- Se han capturado como casos de uso todos los requisitos funcionales
- La secuencia de acciones (flujos de eventos) es correcta, completa y comprensible para cada caso de uso
- Se identifican casos de uso que no proporcionen ningún valor. Si esto ocurre, esos casos de uso deben reconsiderarse, puede ser que sea necesario eliminarlos, o incluirlos como partes de otros casos de uso.

En este reporte figuran todos los actores y casos de uso identificados, cada uno con su documentación, y los diagramas que muestran cómo se relacionan unos con otros.

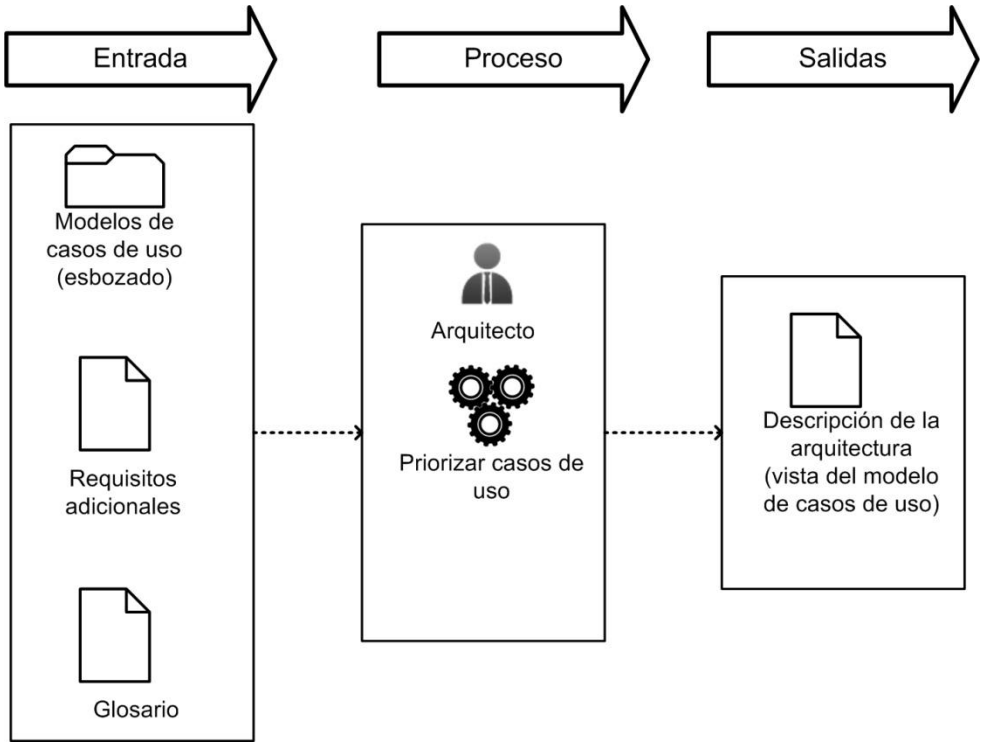
Para esclarecer el panorama, sobre todo a los clientes, es conveniente hacer un resumen al principio del reporte, donde se especifique:

- Cuáles son los casos de uso principales, los que constituyen la razón de existencia del sistema (forman la vista de casos de uso o vista de escenarios).
- Aspectos técnicos importantes sobre el sistema.
- Límites del sistema, especificar claramente las funciones que el sistema no hace, es decir las que quedaron fuera de los límites del sistema.
- Resumen del entorno del sistema, por ejemplo, plataformas de hardware (equipos) y software existente (otros sistemas) o de base (sistemas operativos) y bases de datos.
- El orden (secuencia) en que normalmente se utilizan los casos de uso del sistema.
- Otra funcionalidad no cubierta en el modelo de casos de uso

3.2. Priorización de casos de uso

Los objetivos principales de esta actividad son los siguientes:

- Definir los casos de uso que se van a encarar en primer lugar en las actividades de análisis y diseño a seguir en un desarrollo iterativo.
- Definir un conjunto de casos de uso que representen la funcionalidad central del sistema, aquellos que forman el *core* del sistema.
- Definir los casos de uso que cubre una parte significativa de la arquitectura del sistema, o que ejercitan un punto específico de interés dentro del sistema



Entradas	Trabajador / Proceso	Salidas
Modelo de casos de uso (esbozado)	Arquitecto: Priorizar casos de uso	Descripción de la arquitectura (vista del modelo de casos de uso)
Requisitos Adicionales, si los hubiese		
Glosario		

Para poder priorizar los casos de uso se necesitan:

- El modelo de casos de uso que hemos realizado hasta el momento
- Los requisitos
- Y el glosario que se comenzó a armar.

Con esta información de entradas podemos dedicarnos a priorizar los casos de uso, es decir que estamos decidiendo cuales son los casos de uso que vamos a detallar primero, esto se puede observar en la tabla anterior.

Existen distintos criterios para priorizar los casos de uso, algunos de los cuales son:

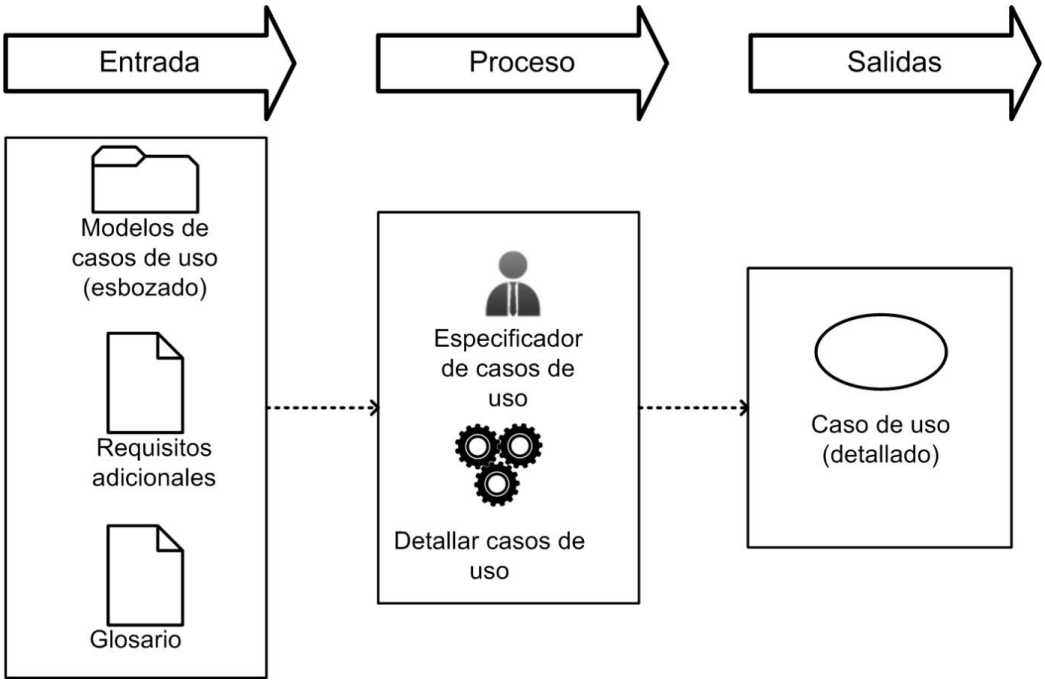
- Casos de uso con alto impacto en la arquitectura del sistema (utilización de recursos, componentes software y hardware, etc.).
- Casos de uso que mitigan riesgos identificados (confiabilidad, disponibilidad, tiempo de respuesta, incertidumbres en general).
- Casos de uso que permiten alcanzar objetivos tácticos (por ejemplo: demo a los usuarios finales).

Los casos de uso priorizados se documentan en diagramas de casos de uso y en un documento de arquitectura, como se vio en el apartado 2.8.

3.3. Especificación de casos de uso

El objetivo principal de esta actividad es describir en detalle los flujos de eventos y demás propiedades de cada caso de uso, de manera tal que tanto los clientes como los desarrolladores los entiendan (ver apartado 2.2). Se hace énfasis nuevamente en que los casos de usos se deben detallar lo más posible, para de esta manera los desarrolladores y quién escribe el manual de los usuarios tienen mayor información.

Para poder especificar un caso de uso se debe tener el esbozo de los casos de uso, los requisitos y el glosario, con esta documentación el responsable de especificar los casos de uso podrá realizar la tarea de especificación obteniendo los casos de usos detallados.

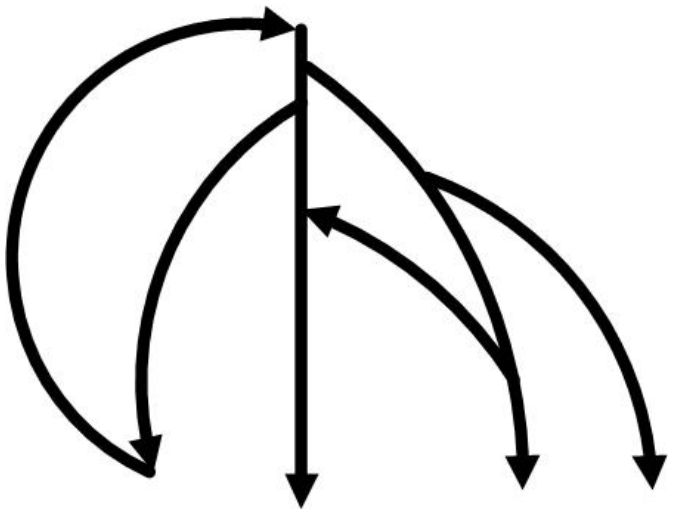


Entradas	Trabajador / Proceso	Salidas
Modelo de casos de uso (esbozado)	Especificador de casos de uso: Detallar los casos de uso	Casos de uso detallados
Requisitos Adicionales, si los hubiese		
Glosario		

La documentación detallada de un caso de uso comprende las sub-actividades detalladas en los puntos 3.3.1. al 3.3.4.:

3.3.1. Documentación de los flujos de eventos

Cada caso de uso normalmente tiene un flujo principal de eventos y una serie de flujos alternativos (uno o más). La estructura de estos flujos de eventos, se puede representar gráficamente como se muestra en la figura.



Representación gráfica de los flujos de eventos de un caso de uso. La flecha negra que está en el centro, corresponde al flujo principal, es aquel flujo que debe ocurrir en la ejecución normal de un caso de uso, mientras que las curvas representan los flujos alternativos, es decir que el caso de uso por algún motivo no puede cumplir con su flujo normal. Algunos flujos alternativos pueden retornar al principio del caso de uso, mientras que otros pueden terminarlo.

El flujo principal representa la secuencia de acciones que hace el actor y el sistema para llevar a cabo la funcionalidad normal o principal definida en el caso de uso. Los flujos alternativos representan el comportamiento de carácter opcional o excepcional, e incluso variaciones del comportamiento normal.

Tanto el flujo principal como los alternativos se deben estructurar como una secuencia de pasos que relatan qué hace el actor y qué hace el sistema en cada uno de ellos.

Ejemplo: supongamos que se tiene un caso de uso “Crear cuenta de usuario” cuyo objetivo es que un actor Usuario pueda registrarse en un sistema. Los flujos de eventos podrían ser:

3.3.1.1. Flujo principal

1. El usuario ingresa sus datos (nombre y apellido, mail, teléfono, nombre de usuario y clave de acceso deseada)
2. El sistema valida los datos y registra al nuevo usuario en su base de usuarios.

3.3.1.2. Flujos alternativos

Alternativa al paso 2: El nombre de usuario ya existe
2.1 El sistema notifica al usuario y le da la posibilidad de indicar un nuevo nombre
2.1 El sistema notifica al usuario que la cuenta de e-mail ya existe y sugiere que ingrese otra.

Alternativa al paso 2: Faltan ingresar algunos datos obligatorios.
2.3 El sistema muestra al usuario los datos que son obligatorios y le da la posibilidad de indicarlos

Observaciones: los datos considerados obligatorios son: nombre y apellido, mail, nombre de usuario y clave de acceso deseada.

Una buena práctica es numerar los pasos de los flujos de eventos, de manera de

poder establecer referencias dentro del mismo documento. Es decir que si dentro de un paso debemos regresar o saltar alguna parte, sea fácil referenciarla.

En la documentación de los flujos alternativos, es de especial importancia contemplar todas las condiciones de error que puedan ocurrir en un caso de uso, e incluso es recomendable especificar los mensajes de error que el sistema deba dar al usuario.

Importante: Una documentación prolija de los flujos alternativos permite que se puedan generar los casos de prueba del sistema, los manuales de usuario, y las ayudas en línea directamente desde los casos de uso.

Por último, también es necesario incluir en los flujos de eventos una descripción de cómo y cuándo comienza el caso de uso, y cómo y cuándo termina.

Ejemplo: dentro del flujo de eventos se puede incluir un título "Activación" para contemplar el inicio.

3.3.1.3. Flujo de eventos

Activación

El usuario selecciona la opción "Generar" del submenú "Orden de compra".

En el flujo principal, y en los flujos alternativos en que haga falta, se puede indicar el momento en que termina el caso de uso.

Flujo principal

1. El sistema presenta una lista de los sectores de origen.
2. El sistema presenta una lista de los sectores destino.
3. El usuario selecciona el sector origen.
4. El usuario selecciona el sector destino.
5. El usuario presiona "Agregar insumos" llamando al caso de uso "Seleccionar insumos".
6. El sistema carga una grilla con los datos seleccionados.
7. El usuario presiona "Aceptar".
8. El sistema guarda los datos de la nueva orden de compra.

Algunos criterios a tener en cuenta:

- En la documentación de los flujos de eventos, hay que concentrarse en documentar QUÉ hace el actor y QUÉ hace el sistema, evitando pensar en COMO lo hacen.
- Si un flujo de eventos se torna demasiado extenso o complejo, o si se descubre que existen partes que son totalmente independientes de otras, es recomendable dividir el caso de uso en dos. Como criterio general se puede considerar como extenso a un flujo de eventos de más de 10 o 12 pasos.
- A medida que se escribe el texto, hay que referirse al glosario, verificando la consistencia de los términos utilizados. Se pueden agregar nuevos términos si surgen durante la especificación, pero hay que tener especial cuidado en modificar los términos existentes, ya que puede impactar en otras partes del modelo.

3.3.2. Documentación de los requerimientos especiales

Esta sub-actividad implica la documentación de cualquier requerimiento que esté vinculado con el caso de uso y no esté mencionado en los flujos de eventos. Usualmente este tipo de requerimientos son no funcionales e impactan en las actividades posteriores de análisis y diseño del sistema.

Estos requerimientos se pueden documentar con texto libre, o texto organizado en categorías, como por ejemplo: Usabilidad, Confiabilidad, Rendimiento, etc.

Alternativamente, pueden no documentarse en cada caso de uso y utilizar alguna de las demás alternativas tratadas en el apartado 2.6 (Especificaciones suplementarias).

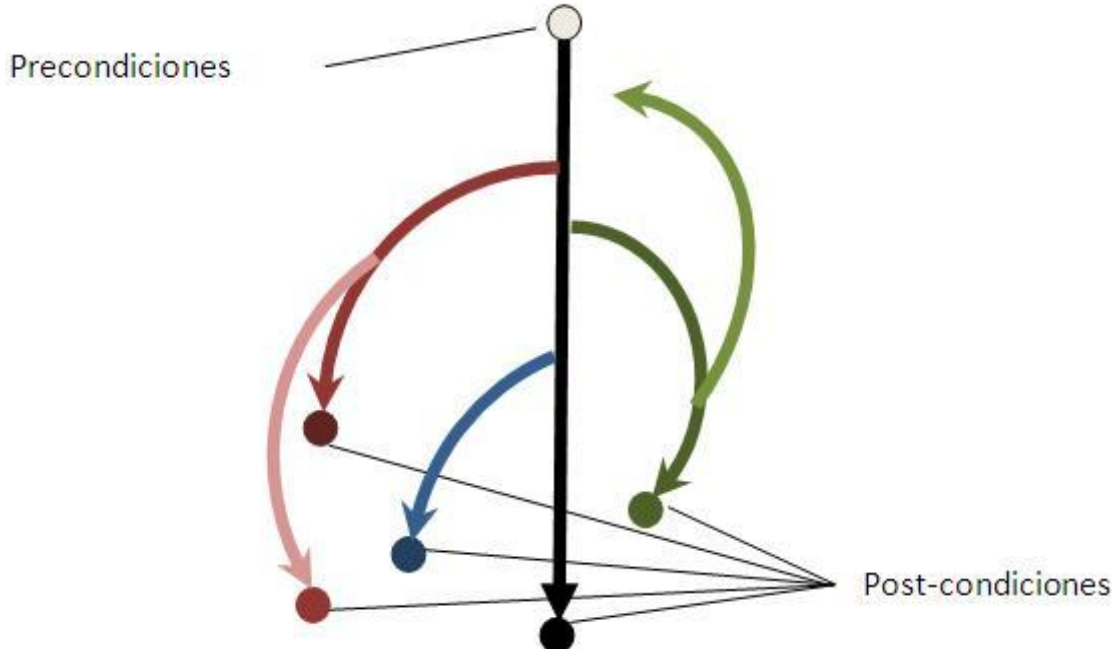
En este material recomendamos que se utilice la misma plantilla que se usa para documentar el caso de uso y se llene el campo "Requisitos especiales", esta plantilla ya se mostró en el punto 2.2.1 del capítulo anterior y se detallará en el punto 3.3.7 de este capítulo. Al usar la misma plantilla que se utiliza para documentar el caso de uso, nos permite tener toda la información junta en un solo lugar.

3.3.3. Documentación de las precondiciones y post-condiciones

Las precondiciones y post-condiciones de un caso de uso se relacionan con el comienzo y la finalización de la ejecución del mismo. La figura siguiente muestra gráficamente este concepto.

En la siguiente figura se representa en forma gráfica las precondiciones y post-condiciones de un caso de uso.

- Las precondiciones: reflejan el estado en el que debe estar el sistema y su entorno para que pueda comenzar la ejecución del caso de uso. Una precondición es una condición (o varias) que se debe cumplir para que el caso de uso pueda dar comienzo. No se deben confundir con el evento que dispara la ejecución del caso de uso, denominado usualmente **Activación.**
- Las post-condiciones reflejan el estado en el que queda el sistema y su entorno luego de la ejecución del caso de uso. Una post-condición es una condición que se cumple luego de la ejecución del caso de uso.



El cumplimiento o no de la condición puede depender del flujo alternativo que se haya seguido. Potencialmente podría haber una post-condición para cada uno de los flujos de eventos, tanto principales como alternativos, como se ve en la figura anterior

Ejemplos:

Se pueden utilizar las precondiciones para indicar condiciones de seguridad que deba cumplir el usuario para poder utilizar un caso de uso.

Precondiciones de este tipo pueden ser: “Existe al menos una empresa de transporte cargada en el sistema y que no haya deuda o envíos pendientes”, “El usuario está logueado en el sistema”, “El usuario tiene permiso para cargar documentos”, o “El usuario pertenece al área que posee el documento”

Las post-condiciones resultan útiles para resumir los resultados alcanzados con la ejecución del caso de uso.

En un caso de uso “baja de empresa de transporte”, la post-condición podría ser: La empresa de transporte fue dada de baja” o en caso de fracaso del caso de uso “la empresa permanece cargada.

En un caso de uso “Crear cuenta de usuario” en el cual un usuario puede registrarse en el sistema, una post-condición posible puede ser “Luego de ejecutado el Flujo principal, en el sistema queda almacenada la nueva cuenta de usuario, y el usuario queda logueado al sistema”.

3.3.4. Documentación de puntos de extensión

Los puntos de extensión que se pueden dar por extensión (*extend*) o por inclusión (*include*), dan la posibilidad de indicar dentro de un caso de uso, los lugares en los que opcionalmente se puede disparar la utilización de otro caso de uso. Este concepto está directamente vinculado con la relación de extensión entre casos de uso (ver apartado 2.4, Diagramas de casos de uso).

La documentación los puntos de extensión consiste en indicar en qué paso de los flujos tanto principales como alternativos, se puede disparar la ejecución de otro caso de uso.

Ejemplo: en un modelo de casos de uso se tiene que el caso de uso “Realizar baja de empresa de transporte” se podría extender al caso de uso “Verificar pedidos pendientes de envío”. Dentro del caso de uso “Realizar baja de empresa de transporte” se documenta:

Puntos de extensión

Realizar baja de empresa de transporte: en el paso 2 del flujo alternativo, mientras está viendo los detalles de la empresa, el usuario puede seleccionar la opción “Verificar pedidos pendientes de envío” disparando dicho caso de uso.

3.3.5. Documentación de observaciones y datos

A menudo resulta de utilidad documentar en una sección separada, los datos específicos que se intercambian en los pasos de cada flujo de eventos y otras observaciones en general.

Ejemplo: supongamos que en un paso del flujo de eventos principal se menciona en el paso 2: Se carga una página con una lista de las empresas de transporte existentes en el sistema y se selecciona la empresa a borrar

En la sección Observaciones y datos se podrían especificar:

1. Los datos de la empresa son: nombre, dirección, y teléfono.

Estos datos resultan útiles a la hora de identificar clases de análisis y/o atributos de las mismas.

3.3.6. Formato de la especificación

El formato más difundido para la especificación de los casos de uso es el siguiente:

- Nombre

- Descripción
- Flujo de eventos
- Precondición
  - Activación
  - Flujo principal
  - Flujos alternativos
- Requerimientos especiales
- Post-condiciones
- Puntos de extensión

Algunas alternativas de formato pasan por las siguientes variaciones:

a) Discriminar los flujos alternativos en flujos de excepción y flujos de variación.

En los flujos de excepción se documentan solamente las condiciones de error del caso de uso, mientras que en los de variación se documentan las distintas formas que tiene el actor de efectuar un mismo paso en el flujo principal.

El formato es el mismo, pero la sección Flujos alternativos está dividida en

3.3.7. Plantilla recomendada

Nombre:		**Nro**:
Actores:		**Versión**:
Objetivo:		
Precondiciones:		
Flujo Principal		**Flujo Alternativo**
1.		
2.		
3.		
4.		
5.		
6.		
7.		
Asociaciones de Inclusión:		
Asociaciones de Extensión:		
Post-condiciones:	**Éxito:**	
	Fracaso:	
Requerimientos Especiales:		
Observaciones:		

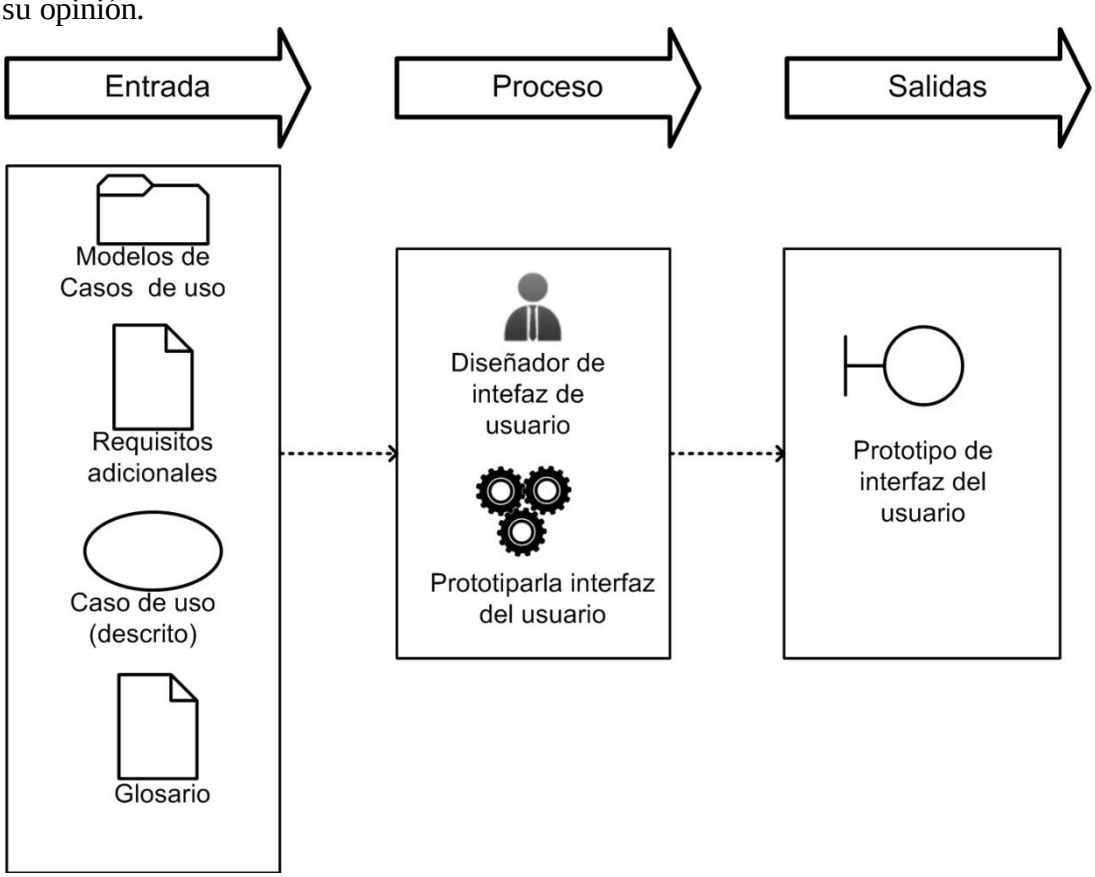
En resumen, la especificación de los casos de uso constituye un contrato que detalla lo que el sistema debe hacer, mostrando cómo es del funcionamiento del mismo. Por lo tanto, se debe tratar de mantener la simplicidad de estos documentos, utilizando el lenguaje del cliente, pero sin omitir ninguno de los detalles necesarios para las actividades posteriores de análisis, diseño, implementación y pruebas del sistema.

Además de constituir un contrato, estas especificaciones constituyen una entrada directa para la construcción de los manuales de usuario y el diseño de las pruebas del sistema.

3.4. Prototipado de la interfaz de usuario

El objetivo de esta actividad es estabilizar las especificaciones de los casos de uso utilizando prototipos validados por el usuario. Los prototipos pueden utilizarse cuando no existe una visión clara entre los clientes y los analistas de lo que el sistema debe hacer, o de la forma que los usuarios deben interactuar con el mismo.

Para cada caso de uso se diseñan uno o más prototipos de interfaz, los cuales pueden implementarse y validarse con el cliente, o simplemente mostrarse al cliente para que dé su opinión.



Entradas	Trabajador / Proceso	Salidas
Modelo de casos de uso	Diseñador de interfaz del usuario: Prototipar la interfaz del usuario	Prototipo de interfaz del usuario
Requisitos Adicionales		
Caso de uso descrito		
Glosario		

Los prototipos a utilizar pueden ser de los siguientes tipos:

- Dibujos en papel
- Dibujos hechos con alguna herramienta gráfica
- Prototipos de ventanas ejecutables realizadas con algún lenguaje de programación
- Prototipos de páginas web en HTML

Muchos desarrolladores no prefieren usar prototipos ejecutables o HTML porque

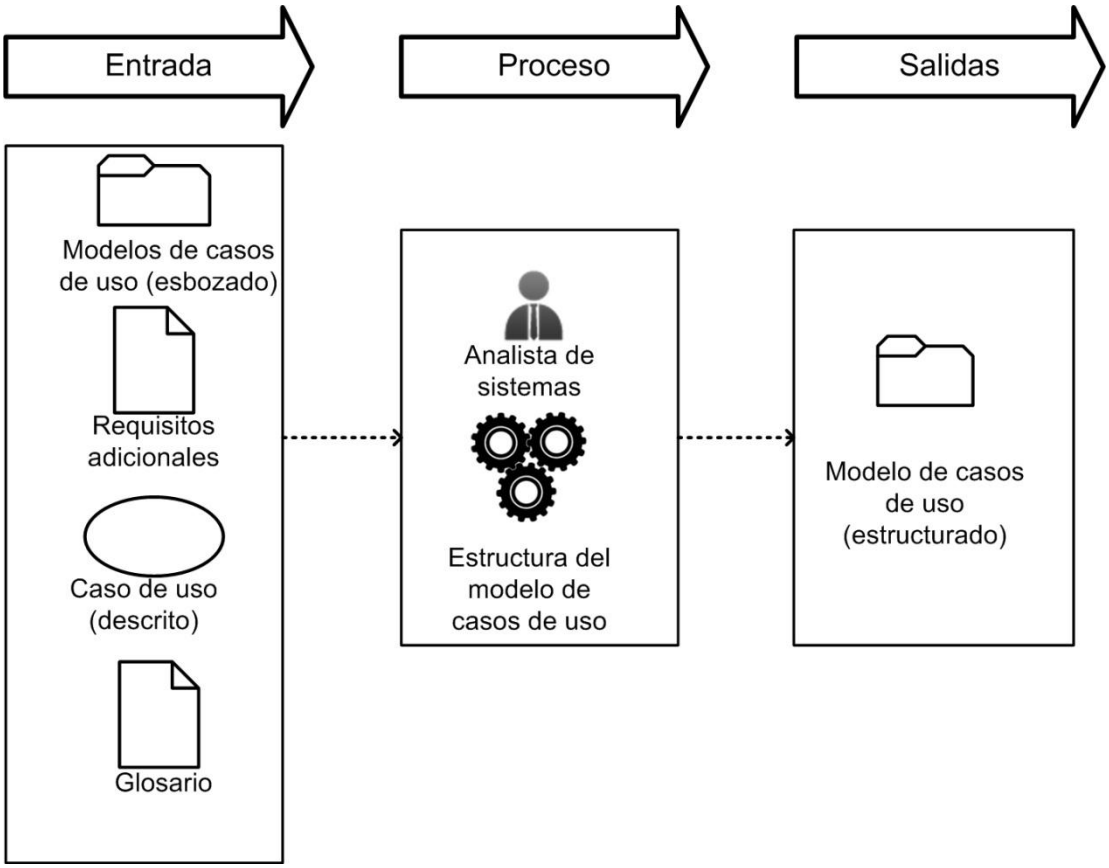
resulta contraproducente, porque los clientes al ver las pantallas que se pueden utilizar, piensan que ya está casi todo hecho y lo que falta es poco para tener el listo el sistema, lo cual no es así. Por ese motivo, muchos recomiendan utilizar dibujos en papel o en herramientas gráficas de dibujo.

3.5. Estructuración del modelo

Los objetivos de esta actividad son:

- Identificar y extraer el comportamiento común entre casos de uso a nuevos casos de uso relacionados con los anteriores a través de extensiones o inclusiones.
- Identificar actores abstractos que definan roles compartidos por otros actores

La estructuración del modelo pasa fundamentalmente por explorar y definir las relaciones que puedan existir entre los distintos elementos que lo componen.



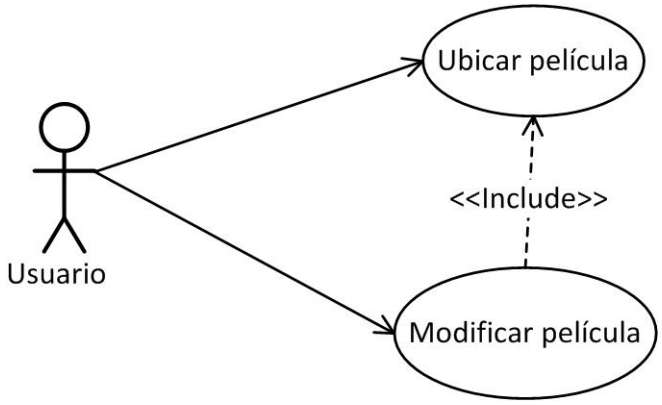
Entradas	Trabajador / Proceso	Salidas
Modelo de casos de uso	Analista de sistemas: Estructurar el modelo de casos de uso	Modelo de casos de uso esbozado
Requisitos Adicionales		
Caso de uso descrito		
Glosario		

3.5.1. Relaciones de inclusión entre casos de uso

Con la información de entrada de que teníamos en el gráfico de especificar los casos

de uso más la salida del mismo se puede proceder a estructurar el modelo. Para ello se utiliza esta relación cuando se detecta que en al menos dos casos de uso diferentes conceptualmente (con funcionalidad distinta) existe alguna secuencia de pasos o flujo de eventos repetidos.

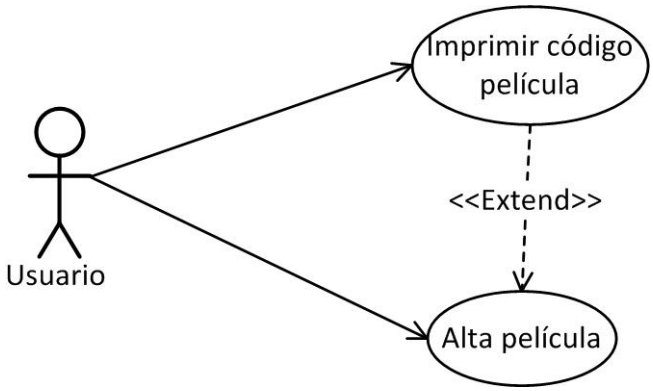
Esa secuencia puede extraerse dando lugar a un nuevo caso de uso relacionado con los otros dos mediante una relación de inclusión.



También se puede utilizar para descomponer un flujo de eventos muy extenso en dos, incluyendo la parte extraída en el caso de uso original mediante esta relación (inclusión).
El concepto fundamental de la relación de inclusión es que un caso de uso, denominado **base,** incluye o contiene como parte de alguno de sus flujos de eventos a otro caso de uso, denominado **inclusión**.

3.5.2. Relaciones de extensión entre casos de uso

Se utiliza esta relación cuando se desea indicar que mientas se está utilizando un caso de uso, se puede utilizar OPCIONALMENTE otro caso de uso, en algún punto de la ejecución del primero.



Dentro de un caso de uso, algunos flujos alternativos pueden representar un comportamiento opcional seleccionado por el usuario. Cuando ese comportamiento puede ser a su vez utilizado por separado, resulta conveniente extraerlo a un nuevo caso de uso relacionado como extensión al caso de uso original.

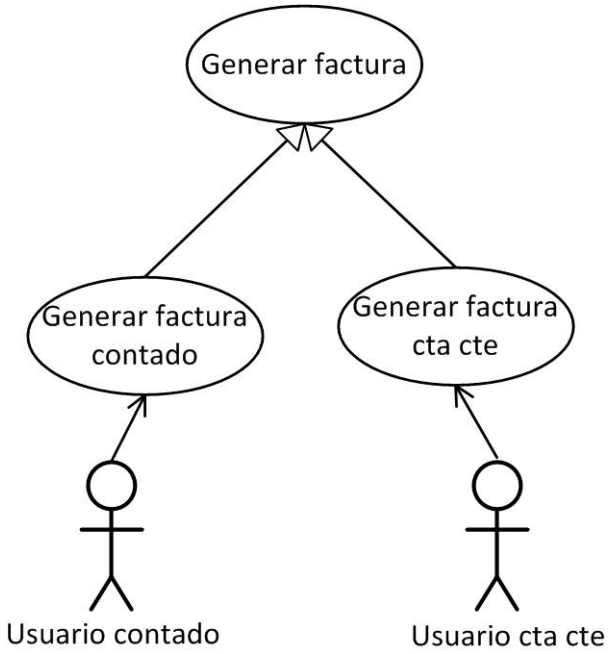
Por ejemplo en el caso de uso “Dar de baja empresa de transporte” el usuario en el paso 2.1, podía seleccionar la opción de consultar los pedidos pendientes de envío. Y este caso de uso ya existía, porque en otra parte del sistema se podía consultarlo.

El concepto fundamental de la relación es que un caso de uso, denominado **base,** puede ser enriquecido o extendido con comportamientos adicionales definidos en otro caso de uso, denominado **extensión**.

3.5.3. Relaciones de generalización entre casos de uso

Esta relación se puede utilizar cuando se tienen casos de uso con funcionalidad muy parecida y con ligeras variaciones en su comportamiento general o en sus flujos de eventos.

Cuando este es el caso, se puede definir un caso de uso denominado ***padre*** donde se condensen todos los factores comunes. Este caso de uso se relaciona mediante relaciones de generalización con los casos de uso denominados **hijos** o **derivados** que brindan la funcionalidad específica (compartida + variación). Cada uno de los casos de uso hijos tendrá las variaciones específicas, mientras que el caso de uso padre tendrá el comportamiento común a ambos.



El concepto fundamental de esta relación es el de herencia en orientación a objetos.

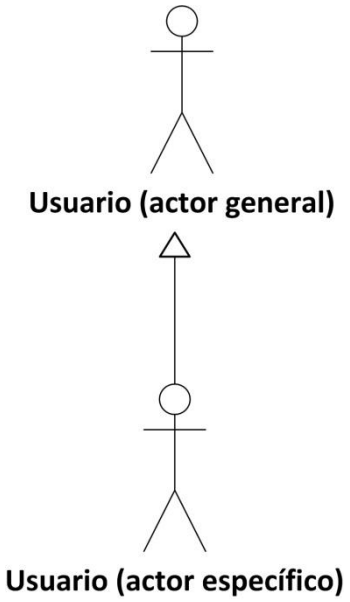
En la práctica, la generalización de casos de uso no se utiliza con frecuencia, sus causas son principalmente a los inconvenientes que trae aparejados en la documentación textual de los casos de uso. No se debe perder de vista, que tanto el caso de uso padre como sus hijos estarán documentados mediante una especificación de caso de uso, y que ante un cambio en el comportamiento de alguno de ellos hay que revisar y posiblemente cambiar los otros dos.

3.5.4. Relaciones de generalización entre actores

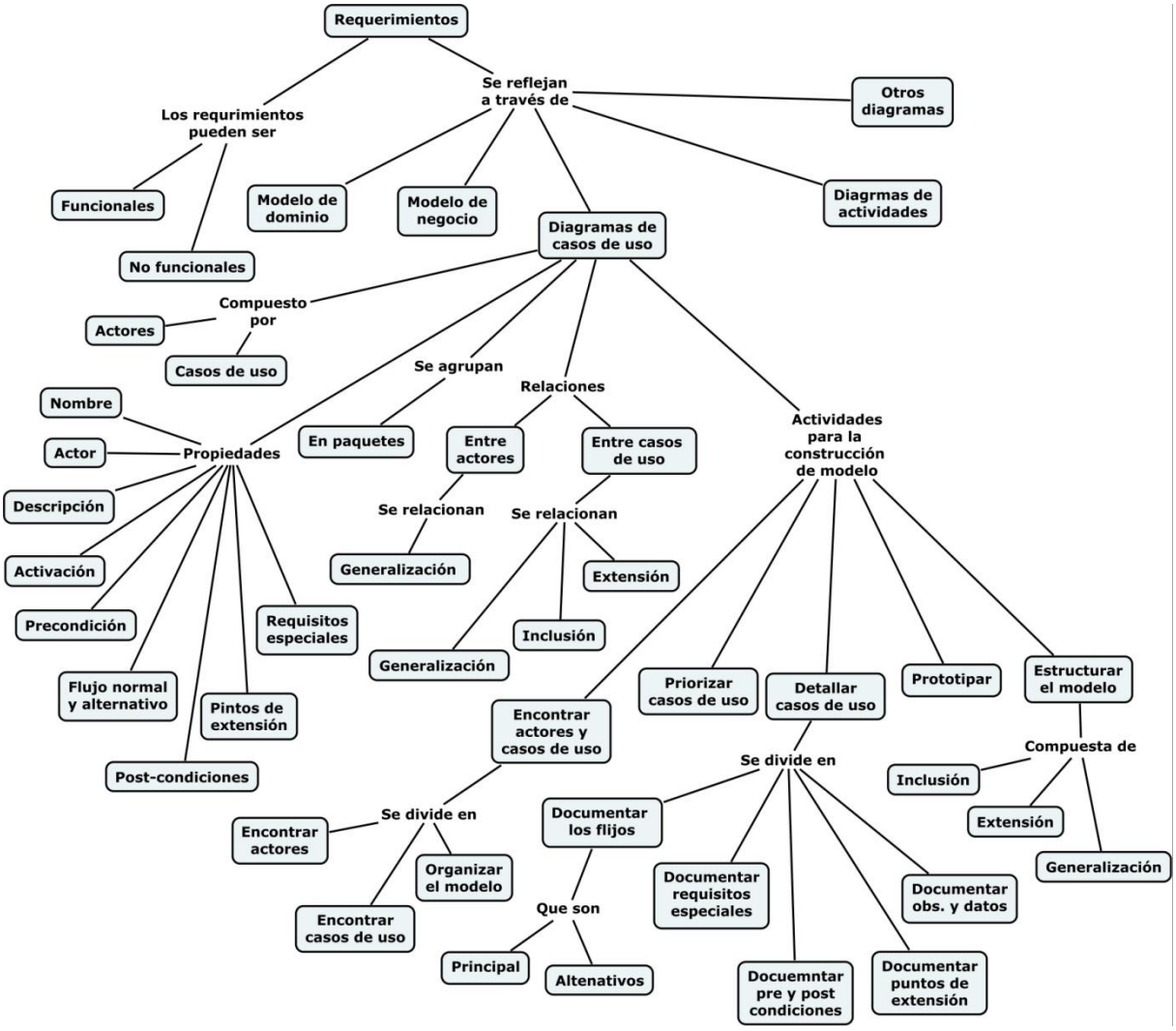
Esta relación se puede utilizar cuando se tienen actores que representan un rol muy parecido pero con ligeras diferencias entre sí.

En este caso, se puede definir un actor denominado padre donde se agrupan los factores comunes. Este actor se **relaciona** mediante relaciones de generalización con los actores denominados **hijos** o **derivados** que definen los roles más específicos.

El concepto fundamental de esta relación es el de herencia en orientación a objetos.



4. Resumen a través de un mapa conceptual



5. Bibliografía

- Peralta, M. (2002), Apuntes de curso de Introducción a UML, Instituto Tecnológico de

Buenos Aires.

- Booch, G, Rumbaugh, J. y Jacobson, I. (2000). El Lenguaje Unificado de Modelado. Addison Wesley, Madrid, España.
- Jacobson, I., Booch, G, y Rumbaugh, J. (2000). El Proceso Unificado de Desarrollo de Software. Addison Wesley, Madrid, España.
- Rumbaugh, J., Jacobson, I. y Booch, G, (2000). El Lenguaje Unificado de Modelado. Manuel de referencia. Addison Wesley, Madrid, España.
- Arlow, J. Neustadt, I. (2005). UML 2. Anaya Multimedia. Madrid, España.